

SAR Ship Detection Project Documentation

Steven Bojilov, Yan Genev, Nadezhda Ivanova, Chahine Med , Anna Piccinini

July 24, 2025

Abstract

This document outlines the SAR Ship Detection Project, aiming to detect and track ships using Synthetic Aperture Radar (SAR) data from sources like Sentinel-1. The core functionality involves machine learning models for land-sea segmentation and image correction, together with deep learning models for ship detection and tracking. The system offers three user interfaces: a Jetson AGX Orin for on-orbit inference and data downlink, a web-based interface for on-demand analysis of SAR data selected via a mAP, and an image upload feature for user-provided raw SAR imagery. Key objectives include developing lightweight, high-accuracy models for real-time ship identification, tracking, and potential applications such as monitoring illegal activities, aiding in search and rescue operations, and general maritime surveillance.

Associated Code Repository: <https://github.com/ioEclipse/SAR-SHIP-DETECTION>

Contents

1	Introduction	4
1.1	Project Overview	4
1.2	Problem Statement	5
1.3	Project Goals and Objectives	6
1.4	Document Structure	7
2	Project Context and Justification	8
2.1	Motivation	8
2.2	Literature Review and Background	9
3	Requirements Specification	12
3.1	Non-Functional Requirements	14
4	System Design	17
4.1	High-Level System Architecture	17
4.2	Detailed Component Design	17
4.2.1	Data Acquisition and Management	17
4.2.2	Band Processing Subsystem	19
4.2.3	Preprocessing Subsystem	20
4.2.4	Machine Learning Detection Model	23
4.3	Data Flow and Workflow	24
5	Implementation and Development	28
5.1	Software and Hardware Environment Setup	28
5.1.1	Training Setup, Hardware + Software	28
5.1.2	Implementation Hardware/Edge	28
5.1.3	Operating System and Core Software Versions	29
5.1.4	Dependency Management and Virtual Environment Setup	29
5.1.5	Accessing the Jetson Nano	29
5.1.6	Essential Command Line Commands	29
5.1.7	RAW Data prediction algorithm (Inference Slicer Method)	30
5.1.8	Google Earth Engine Prediction API	30
5.1.9	AIS Integration	31
5.2	Installation and Setup Guide	32
5.3	Configuration	34
6	Model Training and Optimization	35
6.1	Dataset Preparation Review	35
6.2	Training Process	35
6.3	Optimization techniques for Jetson AGX Orin	37
6.4	Model Deployment Pipeline	38
7	Evaluation and Testing	39
7.1	Test Plan And Methodology	39
7.2	Performance Metrics	40
7.2.1	Detection Accuracy Metrics	40
7.2.2	Inference Efficiency Metrics	40
7.3	Experimental Setup	41
7.4	Evaluation Results and Analysis	42
7.4.1	Quantitative Results	42
7.4.2	Small Vessel Detection Performance	44
7.4.3	False Positive Analysis and Background Interference	44
7.4.4	Operational Threshold Analysis	44
7.4.5	Performance Benchmarking Against Industry Standards	44
7.4.6	Detection Robustness and Sea State Performance	45
7.4.7	Computational Efficiency and Deployment Readiness	45
7.4.8	Limitations and Areas for Improvement	45

8	Future Work and Enhancements	46
9	Conclusion	47
10	Team Structure & Expertise Required	48
A	Pre-processing	52
A.1	Linear Scaling	52
A.2	Gamma Correction	52
A.3	Land masking	52
A.3.1	Lee Filter	52
A.3.2	Multi-Stage Denoising	53
A.3.3	Combined Thresholding	53
A.3.4	Morphological Processing	55
A.3.5	Flood Filling	55
A.3.6	Contour Filtering	56
A.3.7	Iterative Refinement & Final Output	56
B	RAW Data prediction algorithm (Inference Slicer Method)	57
C	Google Earth Engine Prediction API	58
D	AIS key operational steps	59
E	Configuration Template	60

1 Introduction

This project develops an end-to-end system for ship and dark vessel detection using Synthetic Aperture Radar (SAR) imagery fused with Automatic Identification System (AIS) data. Unlike optical or manual methods, SAR enables all-weather, day-and-night monitoring, while AI-driven detection ensures automation and scalability. The focus is on identifying both regular vessels and dark ships—those that disable or manipulate AIS transponders, often linked to illicit activities.

To overcome challenges such as speckle noise, sea clutter, and data volume, the system employs machine learning (YOLO-based detection), edge-optimized inference (Jetson/CubeSat), and intelligent AIS cross-matching. The solution aims to provide timely, reliable maritime domain awareness, particularly in high-risk areas like the Central Mediterranean and West African waters, where irregular migration and IUU fishing pose acute humanitarian and security threats. By generating trusted detections, identifying anomalies (e.g., loitering, STS transfers), and enabling real-time operational use, the project sets a foundation for scalable, global maritime surveillance.

1.1 Project Overview

The SAR Ship Detection Project is an intensive, one-month initiative aimed at developing a comprehensive and operational system for the detection and persistent tracking of maritime vessels from Synthetic Aperture Radar (SAR) imagery. While built primarily around data from the Sentinel-1 satellite constellation, the system is designed with the flexibility to integrate other SAR data sources.

The system ingests raw SAR imagery and applies a structured sequence of preprocessing steps, including:

- Land–sea separation algorithms for accurate coastline delineation.
- Speckle noise reduction methods to counter the granular interference common in SAR images.
- Feature enhancement techniques to maximize object visibility.

The preprocessed data is then fed into a machine learning-based detection pipeline, currently using YOLOv11m, optimized for robust recognition of ships of varying sizes, shapes, and orientations.

To ensure adaptability to diverse operational contexts, the project delivers three complementary access modes:

1. **Edge Deployment:** A compact, high-performance version optimized for devices such as the NVIDIA Jetson AGX Orin, enabling real-time detection in remote or space-based settings.
2. **Web Application:** An interactive platform allowing users to define areas of interest directly on a map for near real-time SAR-based monitoring.
3. **Direct Processing Mode:** A feature allowing users to upload raw SAR images for immediate analysis.

These modalities make the system applicable to a broad range of use cases, including maritime surveillance, environmental monitoring, illegal fishing prevention, border patrol, and search and rescue operations.

Conventional ship detection methods, often dependent on optical imagery and human observation, are constrained by daylight availability and weather conditions. SAR technology overcomes these limitations—providing all-weather, day-and-night coverage, making it ideal for persistent ocean monitoring. However, working with SAR data presents key challenges:

- **Speckle Noise:** Obscures small or distant targets.
- **Variable Target Signatures:** Ships differ in size, structure, and orientation, creating inconsistent radar responses.
- **Cluttered Environments :** Coastal zones and busy routes generate background interference from landmasses, ports, and offshore structures.

- **High Data Volumes** : Continuous high-resolution SAR monitoring demands efficient, automated processing at scale.
- **Need for Automation**: The vastness of global maritime areas requires AI-driven solutions for anomaly detection and monitoring.

A vital extension of the project is the integration of Automatic Identification System (AIS) data. AIS provides vessel positions, identification, and route information, forming a trusted reference for legitimate maritime traffic. By cross-referencing SAR-detected objects with AIS data, the system can:

- Validate detections against known vessel locations.
- Track and analyze ship trajectories.
- Identify anomalies such as inconsistencies or unexpected route deviations.

A particular focus is detecting dark vessels—ships that deliberately switch off AIS transponders or transmit false identities to evade monitoring. Such activities are frequently linked to illegal fishing, smuggling, trafficking, and unauthorized entry into restricted waters. The project tackles this through SAR–AIS Fusion:

1. **SAR Imagery Analysis** – Detects all vessels in the imaged area, regardless of AIS status.
2. **AIS Cross-Matching** – Compares each SAR detection with real-time and historical AIS logs.
3. **Anomaly Flagging** – Any ship visible in SAR imagery with no AIS match (within proximity and time tolerance) is flagged as a potential dark vessel.

This methodology ensures the identification of non-cooperative or suspicious vessels, enhancing maritime domain awareness and enabling timely operational intervention.

By combining high-resolution SAR data with AI-powered detection and intelligent AIS integration, the SAR Ship Detection Project offers a scalable, versatile, and resilient maritime monitoring solution. Its ability to detect both compliant and non-compliant vessel, including elusive dark ships, represents a significant advancement in global maritime security, environmental protection, and safety at sea.

1.2 Problem Statement

Irregular migration and illegal, unreported, and unregulated (IUU) fishing create acute operational demands for maritime domain awareness across the Mediterranean and West African waters, where the central Mediterranean remains the deadliest maritime migration route with at least 3,105 lives lost in 2023 alone. West African fisheries face persistent pressure from industrial fleets and IUU activity, undermining food security and livelihoods in countries such as Senegal, where overexploitation and weak monitoring exacerbate stock depletion and conflict with artisanal fleets. These dynamics increase the need for persistent, wide-area, all-weather sensing and timely interdiction support.

Synthetic aperture radar (SAR) data, with its all-weather and day-night imaging capabilities, offers a critical resource for ship detection and tracking; however, its effective utilization hinges on advanced artificial intelligence (AI) integrated with edge computing, particularly on CubeSats, to enable real-time analysis, reduce data transmission burdens, and deliver precise insights.

This work targets a technical gap: accurate, timely detection and tracking of “dark vessels” (ships that disable or manipulate AIS) by fusing spaceborne SAR detections with AIS data and complementary RF/OSINT where available, optimized for edge execution on resource-constrained platforms (e.g., CubeSats) to enable near-real-time surveillance with reduced downlink requirements.

Concretely, the problem is to design, implement, and validate an end-to-end pipeline that:

- Detects vessels in SAR imagery with low false-alarm rates across diverse sea states and backgrounds.
- Associates SAR detections with AIS tracks to flag non-matches and suspicious behaviors indicative of dark activity, including loitering and ship-to-ship (STS) transfers
- Runs efficiently at the edge to prioritize downlink of intelligence rather than raw imagery, enabling faster operational response.

Why Current Solutions Fall Short Current SAR–AIS fusion and dark vessel detection approaches face four main limitations:

1. **SAR Detection Accuracy:** Existing CFAR-based or pretrained deep learning models remain vulnerable to sea clutter, incidence angle variation, speckle noise, and coastal proximity, leading to geolocation offsets and elevated false-alarm rates.
2. **AIS Association Reliability:** Many pipelines lack robust, uncertainty-aware spatiotemporal matching, resulting in missed or false “dark” classifications when AIS data contain gaps, spoofing, or duplicates.
3. **Behavioral Event Detection:** Dark activities such as loitering or ship-to-ship transfers require multi-step fusion of AIS trajectories and satellite detections, but current methods suffer from fragile spatiotemporal modeling and limited large-scale validation.
4. **Operational Readiness at the Edge:** Few solutions are optimized for CubeSats’ constrained compute, memory, and power budgets, forcing downlink of full SAR scenes instead of prioritized detections, which delays actionable intelligence.

These shortcomings lead to slower operational response, higher false positives near coasts, inconsistent dark vessel identification in dense traffic, and limited viability for real-world deployment.

1.3 Project Goals and Objectives

This project is dedicated to the design, implementation, and deployment of a practical ship-detection system based on Synthetic Aperture Radar (SAR). By combining state-of-the-art machine learning techniques with real-world data and edge-computing platforms, our project delivers a comprehensive end-to-end solution tailored for maritime surveillance.

To ensure both technical rigor and operational relevance, we have defined the following high-level goals and SMART objectives:

- **Robust Training Dataset:** Curate and annotate a diverse collection of SAR coverages, including open-sea, coastal, and congested port scenarios, to achieve at least 10,000 labeled vessel instances by Month 6.
- **Lightweight Inference Models:** Develop and train convolutional neural networks optimized for low-power inference, targeting an average per-image inference time under $50 \cdot ms$ on desktop GPUs by Month 8.
- **Edge Platform Optimization:** Identify the optimal neural-network architecture for deployment on the NVIDIA Jetson Nano; implement 8-bit quantization and TensorRT acceleration to sustain real-time processing at $\geq 5 \cdot frames$ per second by Month 10.
- **Reliable Object Detection & Tracking:** Integrate detection and multi-object tracking modules to maintain persistent vessel identities across at least 95% of sequential SAR passes in test scenarios by Month 12.
- **AIS Data Fusion:** Correlate SAR detections with AIS broadcasts to confirm $\geq 90\%$ of vessel matches, resolve at least 80% of ambiguous detections, and flag dark-vessel anomalies for operator review by Month 11.
- **User-Actionable Outputs:** Generate georeferenced alerts, summary reports, and annotated imagery formatted for rapid ingestion by maritime operators, with average report generation times under $10 \cdot seconds$ by Month 12.
- **Multi-Modal User Interfaces:** Deliver three distinct access modalities: a GPU-powered web demonstration, an interactive world-map dashboard, and a local raw-image upload portal to support diverse end-user workflows, all operational by thesis defense.

How These Objectives Solve the Problem

- Better SAR robustness and geolocation improve detection fidelity in precisely the conditions that current systems fail, enabling dependable situational awareness in rough seas and near coasts where migration and IUU risks concentrate.
- Uncertainty-aware AIS matching and data hygiene reduce false “dark” flags and missed matches, making dark-vessel identification trustworthy enough for tasking scarce assets.
- Behavioral analytics turn detections into operational leads (loitering, STS), aligning outputs with interdiction workflows rather than raw imagery review.
- Edge optimization and intelligence-first downlink directly address bandwidth and latency constraints typical of CubeSat-class platforms, enabling timely action rather than post hoc analysis.
- Validation in the Central Mediterranean and West Africa ensures measurable impact where humanitarian and governance stakes are highest, with metrics tied to known threat patterns and operational needs.

1.4 Document Structure

To ensure comprehensive understanding, this documentation is organized into a logical flow mirroring the project’s development lifecycle.

It commences with an **Executive Summary** and **Introduction** for a concise project overview. The **Project Context and Justification** grounds the effort in its broader significance, while **Requirements Specification** meticulously lists all functional and non-functional demands.

The core technical details are captured in **System Design**, explaining both the high-level architecture and individual component designs. **Implementation and Development** provides the essential "how-to" for reproducing the project’s environment and code.

The specifics of machine learning are detailed in **Model Training and Optimization**, leading to the empirical **Evaluation and Testing** of the system’s performance.

A practical **Usage Guide** assists in system operation, and forward-looking sections on **System Optimization**, **Known Issues and Troubleshooting**, and **Future Work** and **Enhancements** discuss continuous improvement, culminating in the **Conclusion**.

2 Project Context and Justification

This project is grounded in the urgent need for robust maritime domain awareness, with Synthetic Aperture Radar (SAR) offering unmatched advantages over optical sensors through all-weather, day-and-night monitoring. While traditional detection methods and early datasets (e.g., CFAR, SSDD, OpenSARShip) advanced SAR-based ship detection, persistent issues such as speckle noise, nearshore clutter, and small vessel detection remain. Recent deep learning approaches (YOLO-based models, polarization exploitation, multi-scale datasets) have significantly improved performance but still face generalization and near-coast detection limitations.

The most pressing challenge is dark vessel detection—identifying ships that deliberately disable or spoof AIS. Emerging solutions since 2018, from commercial SAR–RF fusion (ICEYE, Spire) to global competitions (xView3) and multi-sensor fusion platforms (MDA Space), show progress but continue to struggle with false positives, coastal complexity, and limited validation at scale.

This project builds on these insights by developing a scalable, automated SAR–AIS fusion framework optimized for edge deployment, ensuring timely and reliable detection of both cooperative and non-cooperative vessels, with direct applications to illegal fishing, irregular migration, and global maritime security.

2.1 Motivation

The SAR Ship Detection Project is motivated by the critical need for enhanced maritime domain awareness and the unparalleled advantages of Synthetic Aperture Radar (SAR) in addressing the limitations of conventional surveillance methods. Oceans serve as the backbone of global trade, security, and ecological balance, necessitating reliable and real-time monitoring of maritime activities. SAR technology, with its all-weather, day-and-night imaging capability, provides a unique solution to this challenge, overcoming the constraints of optical sensors that are hindered by cloud cover, darkness, or adverse weather conditions. The significance of automated SAR-based ship detection spans multiple domains, each with substantial real-world implications:

- Maritime Safety & Security
 - Enables search and rescue (SAR) operations by rapidly locating distressed vessels.
 - Detects illegal activities, including piracy, smuggling, and unauthorized fishing, ensuring compliance with maritime laws.
 - Enhances border security by monitoring territorial waters and identifying suspicious vessel movements.
- Defense & National Security
 - Supports naval surveillance by tracking warships, submarines, and other strategic assets.
 - Provides intelligence-gathering capabilities in contested or remote maritime regions.
 - Improves situational awareness in critical waterways (e.g., straits, exclusive economic zones).
- Economic & Commercial Applications
 - Optimizes shipping logistics by analyzing vessel traffic patterns and route efficiency.
 - Aids port management and maritime insurance through real-time vessel tracking.
 - Facilitates trade monitoring by verifying cargo movements and detecting anomalies.
- Environmental & Resource Protection
 - Prevents illegal, unreported, and unregulated (IUU) fishing in protected marine zones.
 - Monitors offshore infrastructure (e.g., oil rigs, wind farms) for security and maintenance.
 - Supports disaster response by assessing ship movements during oil spills or other marine incidents.

Despite these compelling applications, existing ship detection systems often struggle with speckle noise, cluttered environments, and scalability issues, particularly when processing large volumes of SAR data. This project addresses these gaps by developing a robust, automated, and scalable detection framework that leverages deep learning (YOLOv11) to deliver actionable maritime intelligence. By harnessing SAR's unique capabilities, this work aims to bridge the divide between data availability and operational decision-making, ultimately contributing to safer, more secure, and sustainably managed oceans.

2.2 Literature Review and Background

Synthetic Aperture Radar (SAR) ship detection is a cornerstone of maritime surveillance, leveraging its all-weather, day-and-night imaging capabilities to monitor vessels across diverse conditions. Traditional methods, such as Constant False Alarm Rate (CFAR) detectors, have been foundational but often suffer from high false alarm rates in complex backgrounds and struggle to detect small or closely spaced ships. These limitations have driven the adoption of deep learning techniques, which offer superior feature extraction and pattern recognition. Since 2017, significant advancements have been made through innovative algorithms and specialized datasets, addressing challenges like multiscale imbalances, variable ship sizes, and complex environments.

In 2017, [1] introduced an improved Faster R-CNN model tailored for SAR ship detection, utilizing the SAR Ship Detection Dataset (SSDD) with 1,160 images (500×500 pixels, 1–15 m resolution) from Sentinel-1, RadarSat-2, and TerraSAR-X. By incorporating feature fusion, transfer learning, and hard negative mining, their approach achieved higher accuracy and lower computational cost than traditional methods, setting a precedent for deep learning in this domain. Concurrently, [2] developed OpenSARShip 2.0, a dataset with 34,528 Sentinel-1 ship chips (30×30 to 120×120 pixels, 2 m resolution), integrated with AIS information and enhanced with interference labeling and type classification, enabling deeper ship target interpretation.

In 2018, [3] introduced the OpenSARShip dataset, comprising 11,346 Sentinel-1 ship chips (30×30 to 120×120 pixels, 2.2 m resolution), designed as a benchmark for ship interpretation algorithms due to its large scale, diversity, and reliability. Building on this, [4] constructed a SAR dataset with 39,729 ship chips (256×256 pixels, 3–25 m resolution) from 102 Gaofen-3 and 108 Sentinel-1 images, supporting robust object detection in complex backgrounds without land-ocean segmentation.

Similarly, [5] released AIR-SARShip-1.0, with 31 Gaofen-3 images (500×500 pixels, 1–3 m resolution), achieving an average precision of 88.1% using density-connected neural networks, addressing data scarcity in high-resolution SAR imagery.

In 2020, [6] presented LS-SSDD-v1.0, a dataset of 9,000 Sentinel-1 sub-images (800×800 pixels) focused on small ship detection, introducing a Pure Background Hybrid Training mechanism to suppress false alarms, validated through ablation studies.

In 2021, [7] released SRSSDD-v1.0, containing 666 Gaofen-3 images (1024×1024 pixels, 1 m resolution) with 2,844 ships across six categories, tackling rotated frame detection in nearshore environments. Also in 2021, [8] introduced the Dual-Polarimetric SAR Ship Detection Dataset (DSSDD), with 1,236 Sentinel-1 image slices (256×256 pixels, 3,540 ships), proposing a Memory-Augmented Autoencoder that achieved a mean average precision (mAP) of 0.94 by leveraging VV polarization and VH polarization data to reduce false alarms.

Advancing to 2024, [9] proposed YOLO-RC for range-compressed domain detection, achieving an F-score of 88.84% and average precision of 84.09% on a self-built dataset, enhancing real-time capabilities by exploiting amplitude gradients and geometric loss.

Finally, [10] developed an enhanced YOLO network with Shuffle Re-parameterization, Space-to-Depth, and Hybrid Attention modules, achieving AP50 scores of 77.2%, 91%, and 95% on LS-SSDD, HRSID, and Vision-MRSSD datasets, respectively, significantly improving small ship detection.

Despite these advancements, critical gaps remain, including the limited inclusion of VV polarization and VH polarization data in datasets like SSDD and OpenSARShip, which hinders

differentiation of ships from background clutter. Additionally, the restricted resolution range in many datasets limits model generalization across operational conditions, and challenges persist in detecting small ships in complex nearshore environments.

Future research should prioritize diverse datasets with multi-polarization data, broader resolution ranges, and advanced feature extraction techniques to enhance detection accuracy and robustness, paving the way for more reliable SAR ship detection systems.

These details are organized in Table 1 to provide a clearer comparison.

Author & Year	Method	No. of Images & Shapes	Sensors	Resolution
Li et al. (2017) [1]	Improved Faster R-CNN	1,160 (500×500)	Sentinel-1	1–15 m
Huang et al. (2018) [3]	YOLOv8	11,346 (30×30 to 120×120)	Sentinel-1, Gaofen-3	2–22 m
Wang et al. (2019) [4]	Modified SSD	43,819 (256×256)	Gaofen-3, Sentinel-1	3–25 m
Sun et al. (2019) [5]	Dense connected network (DCENN)	1,116 (500×500)	Gaofen-3	1–3 m
Wei et al. (2020) [11]	CNN-based detection/segmentation	5,604 (800×800)		0.5–3 m
Zhang et al. (2020) [6]	Pure Background Hybrid Training (PHBT)	9,000 (800×800)	Sentinel-1	10 m
Lei et al. (2021) [7]	Rotated detectors, CTA-based	666 (1024×1024)	Gaofen-3	1 m
Hu et al. (2021) [8]	R3Det and YOLOv4 with MemAE	1,236 (256×256)	Sentinel-1	10 m
Tan et al. (2024) [9]	YOLO-RC	HRSID and SSDD	Sentinel-1, TerraSAR-X, TanDEM-X	0.5–15 m
Tang et al. (2024) [12]	DBW-YOLO (YOLOv7-tiny)	5,604 (HRSID), 1,160 (SSDD)	Sentinel-1, TerraSAR-X, TanDEM-X	0.5–15 m
Sun et al. (2025) [10]	MSDFF-Net	R-SSDD, R-HRSID	Sentinel-1, TerraSAR-X, TanDEM-X	0.5–15 m
Guan et al. (2025) [13]	Enhanced YOLO (SR module)	HRSID and iVision-MRSSD-MRSSD	Sentinel-1, TerraSAR-X, TanDEM-X	0.5–15 m

Table 1: Summary of Public Datasets and Deep Learning Methods for SAR Ship Detection

Dark Vessel Detection and Maritime Surveillance Challenges The emergence of "dark vessels", vessels that deliberately disable their Automatic Identification System (AIS) transponders or engage in spoofing to evade detection, has introduced critical challenges to maritime surveillance systems. These non-cooperative vessels pose significant threats to global maritime security, environmental protection, and fisheries management, requiring advanced detection methodologies beyond traditional AIS-dependent approaches.

Dark vessel detection has gained substantial momentum since 2018, when [14] ICEYE and Spire Global announced collaborative SAR-based technologies to enable global monitoring of dark vessels engaged in illegal activities at sea. This partnership addressed the estimated \$23 billion annual economic impact of illegal fishing activities, demonstrating the urgent need for comprehensive surveillance solutions. The approach combined SAR imagery with radio frequency (RF) analysis to identify vessels operating without proper AIS transmissions.

In 2019, [15] highlighted that SAR's primary advantage lies in detecting vessels not transmitting AIS signals, particularly in monitoring illegal fishing operations where crews commonly disable or manipulate AIS data to provide false coordinates. This capability became increasingly vital as maritime authorities recognized that illegal fishing vessels frequently employed AIS manipulation tactics to evade enforcement efforts.

The 2021 xView3 Challenge, organized by the Defense Innovation Unit (DIU) and Global Fishing Watch, marked a pivotal development in dark vessel detection research. This international competition attracted 2,000+ participants from 67 countries, focusing on developing machine learning algorithms to detect and characterize dark vessels in SAR imagery. The challenge utilized the xView3-SAR dataset containing 243,018 labels across 991 images, representing one of the largest open-source maritime SAR datasets for vessel detection research.

The top-performing xView3 solutions achieved aggregate F1 scores ranging from 0.5717 to 0.6177, with the first-place "BloodAxe" model attaining 0.7702 for detection (F1D) and 0.9392 for vessel classification (F1V). However, shore-proximity detection remained challenging, with F1S scores between 0.4205 and 0.5310 across top performers. These results highlighted significant performance gaps in complex nearshore environments where dark vessels frequently operate.

In 2022, [16] Starboard Maritime Intelligence's analysis of Tuvalu's 750,000 km² Exclusive Economic Zone revealed substantial detection discrepancies between SAR and RF technologies. Their study found that RF sensors produced significantly more dark detections than SAR, with many potential false positives attributed to non-AIS transmitting sources including military vessels, yachts, and aircraft radar systems. This highlighted the critical need for multi-sensor validation approaches in dark vessel detection.

Recent developments in 2024 have focused on multi-temporal detection approaches [17]. Perivallon implemented multi-temporal dark vessel detection using Sentinel-1 and AIS data correlation, demonstrating improved tracking capabilities for non-cooperative vessels through temporal analysis patterns. Similarly, [18] introduced automated detection of dark ship-to-ship transfers using deep learning and PlanetScope satellite imagery, identifying over 400 illicit transshipment events in the Kerch Strait since 2022.

Advanced fusion approaches have emerged to address detection limitations. [19] MDA Space's Dark Vessel Detection platform integrates SAR data with space-based optical, RF, and radar data through machine learning algorithms. This multi-sensor approach has been deployed globally, including supporting Ecuador's efforts to monitor illegal fishing around the Galapagos Islands and throughout the Indo-Pacific region.

A summary of the issues of these approaches is organized in Table 2.

Author & Year	Reported Performance & Key Capabilities	Quantified Issues / Gaps	Metrics to Improve
Li et al. (2017) [1]	Higher accuracy and lower cost than CFAR; feature fusion, transfer learning, hard negative mining	Missed small/closely spaced ships; single-pol (VV only)	AP for small ships; false alarm rate; polarization fusion
Huang et al. (2018) [3]	AIS integration; interference labels; ship type classification	No multi-pol; limited coastal complexity	Polarization use; nearshore detection F1
Wang et al. (2018) [4]	Works without land-ocean segmentation	Missed small ships in high clutter	Recall for small vessels in clutter
Zhang et al. (2020) [6]	Improved small ship detection via pure background hybrid training	AP for small ships not reported; high nearshore FPs	F1 for small ships; shore clutter rejection
Lei et al. (2021) [7]	Supports rotated frame detection; oriented bounding boxes	Low nearshore recall	Nearshore F1; rotation handling
Hu et al. (2021) [8]	mAP = 0.94; uses VV & VH polarization	Limited size diversity; few coastal samples	Multi-scale AP; coastal robustness
Tan et al. (2024) [9]	F-score = 88.84%; AP = 84.09%; real-time capability	Underperforms in cluttered scenes	Real-time F1 in complex backgrounds
Guan et al. (2025) [13]	AP50: 77.2%, 91%, 95% (LS-SSDD, HRSID, Vision-MRSSD); small ship modules	No polarization fusion; dataset-specific tuning	Cross-dataset generalization; multi-pol AP
ICEYE + Spire (2018)	RF + SAR fusion for AIS-off vessels	Validation gaps; no open benchmark	Publicly benchmarked F1 for dark vessels
Mongabay (2019)	Operational SAR monitoring of illegal fishing	No quantitative metrics; misses small AIS-off vessels	Quantified recall for AIS-off small targets
xView3 Challenge (2021)	F1D: 0.7702; F1V: 0.9392; nearshore F1S: 0.4205–0.5310	≈30% drop in nearshore detection	Shore-proximity F1; small-vessel detection
Starboard Maritime Intelligence (2022)	RF detections > SAR detections in Tuvalu EEZ	High RF false positives; SAR undercount	FP reduction; SAR+RF fusion accuracy
Perivallon & MDA Space (2024)	Multi-temporal SAR + optical + RF + AIS fusion	Limited small-ship validation; high processing complexity	Multi-temporal small-ship recall; fusion efficiency

Table 2: Summary of Current SAR Ship and Dark Vessel Detection Approaches, Quantified Gaps, and Target Metrics for Improvement

3 Requirements Specification

The system requirements define the functional, non-functional, and dark-vessel-specific capabilities necessary for a robust SAR-based ship detection platform. Core features include user inputs (selection of geographic areas, dates, and custom SAR uploads), detection outputs (images with bounding boxes, per-ship exports, JSON metadata), and processing functions (land masking, speckle filtering, polarization handling). The system is designed to be fault-tolerant with clear error reporting, queue management, and downloadable results.

For dark vessel detection, the system must integrate AIS fusion, spatial correlation, anomaly analysis, and alerting to identify ships not broadcasting AIS or showing suspicious behavior (e.g., disabling AIS, ship-to-ship transfers). Multi-source validation (SAR, AIS, RF, OSINT) is emphasized to minimize false alarms.

Non-functional requirements ensure accuracy ($\geq 85\%$ mAP), real-time performance (< 40 ms inference), scalability (5 users concurrently), user-friendliness, security, and extensibility. Special focus is placed on edge optimization (Jetson hardware), logging, modularity, privacy compliance, and visualization of AIS vs. dark detections with confidence levels. Collectively, these requirements ensure the system is operationally viable, scalable, and trustworthy for global maritime surveillance and dark vessel monitoring.

Table 3: Functional Requirements for Ship Detection AI System

Requirement ID	Title	Description
REQ-SYS-OUT-001	Output Details	The system shall output ship bounding boxes, class labels, and unique tracking IDs.
REQ-USER-INPUT-002	User Selection	Users shall be able to select coordinates (longitude and latitude), date, and size of scanned area on the provided map (Google Earth map) where the user wants the AI to detect ships.
REQ-USER-INPUT-003	Input SAR	The system shall allow the user to input their own SAR imagery, and the AI will look for ships in it.
REQ-SYS-DOWNLOAD-004	Download Full Image	The system shall allow the user to download the whole image with the detected ships in bounding boxes as a .jpg format.
REQ-SYS-DOWNLOAD-005	Download Single Ship	The system shall allow the user to download each ship visible on the provided image as a separate .jpg format image by inputting the ID of the desired ship.
REQ-SYS-DOWNLOAD-006	Download JSON Data	The system shall allow the user to download a JSON file with the bounding box coordinates, width, class label, and unique ID of each ship.
REQ-SYS-PROC-007	Ignore Land	The system shall not take land into account while processing, but land should still be visible in the output image.
REQ-SYS-PROC-008	Image Preprocessing	The system shall remove speckles and enhance the image during preprocessing, but these effects should not apply to the final image.
REQ-SYS-PROC-009	Polarization Choice	The system shall choose another polarization band if the VV polarization polarization is not an option.
REQ-SYS-DET-010	Ship Detection	The system shall detect only ships in SAR imagery.
REQ-SYS-DET-011	Respond & Display	The system shall respond to user requests to detect ships at a desired location and provide an image showing ships in bounding boxes with class labels and unique tracking IDs.
REQ-SYS-DET-012	Avoid Confusion	The system shall not get confused by non-ship objects.
REQ-SYS-DET-013	Any Polarization	If the given image is not in the VV polarization polarization band, the system shall still detect ships.
REQ-SYS-REPORT-014	No Ships Report	The system shall not display or upload any image if no ships are found, and shall output a system report.

Requirement ID	Title		Description
REQ-SYS-REPORT-015	Format Error		The system shall report an error for an incompatible format upload.
REQ-SYS-REPORT-016	Bad Connection		If a user takes too much time to send or receive data, the process will stop and an error will be returned.
REQ-SYS-REQUEST-017	User Queue		Upon request, the user shall be sent to a queue and will wait for the server to begin processing.
REQ-DARK-AIS-001	AIS Data Integration		The system shall integrate AIS data to cross-verify detected ships with AIS broadcasts to identify vessels that turn off or manipulate AIS signals (i.e., dark vessels).
REQ-DARK-DET-002	Dark Vessel Detection		The system shall detect vessels not broadcasting AIS signals by leveraging SAR imagery features indicative of dark vessels and flag suspicious detections.
REQ-DARK-LOC-003	Location Matching		The system shall spatially correlate SAR detections with AIS pings within a 1 km radius to identify discrepancies indicating potential dark vessel activity.
REQ-DARK-ANAL-004	Suspicious Analysis	Activity	The system shall provide analytics and reports on potential dark vessel behaviors, such as patterns of AIS disabling and ship-to-ship transfers detected from multi-temporal SAR images.
REQ-DARK-INPUT-005	External Support	Dataset	The system shall support importing historical and real-time AIS datasets to enhance dark vessel detection accuracy and verification.
REQ-DARK-REPORT-006	Dark Vessel Alerts		The system shall generate alerts when dark vessel activity is detected, including time, location, and confidence scores based on integrated SAR and AIS data analysis.
REQ-DARK-VALID-007	False Alarm Mitigation		The system shall minimize false alarms in dark vessel detection by incorporating multi-source validation and context-aware filtering of non-vessel artifacts.

3.1 Non-Functional Requirements

Table 4: Non-Functional Requirements for Ship Detection AI System

Requirement ID	Title	Description
REQ-NFP-ACC-001	AI Accuracy	The detection model shall achieve a Mean Average Precision (mAP) of at least 85% on the dataset.
REQ-NFP-ENV-002	Environment Constraints	The system shall operate within the specified power constraints of Jetson Nano.

Requirement ID	Title	Description
REQ-NFR-REL-003	Error Handling and Reporting	The system shall gracefully handle unexpected inputs (e.g., corrupted SAR files, invalid coordinates) and provide informative error messages or system reports, including for scenarios where no ships are found.
REQ-NFR-REL-004	Data Integrity	The system shall ensure the integrity of processed SAR imagery and generated output files (e.g., bounding box JSON, detected ship images) against corruption during storage and download.
REQ-NFR-USABILITY-005	Intuitive User Interface	The web application user interface shall be intuitive and easy to navigate for users with basic computer proficiency, enabling them to select areas of interest and download results with minimal training.
REQ-NFR-USABILITY-006	Feedback Mechanism	The system shall provide clear visual feedback to the user during long-running operations (e.g., "processing in progress," "downloading image").
REQ-NFR-MAINT-007	Code Modularity	The system's codebase shall be modular and well-documented to facilitate future enhancements, bug fixes, and integration of new algorithms or data sources.
REQ-NFR-MAINT-008	Logging and Monitoring	The system shall implement comprehensive logging of critical operations and errors to facilitate troubleshooting, performance monitoring, and auditing.
REQ-NFR-MAINT-009	Configuration Management	System parameters and model configurations shall be externalized and easily manageable without requiring code changes or redeployment.
REQ-NFR-COMPAT-010	Browser Compatibility	The web-based user interface shall be compatible with the latest versions of major web browsers (e.g., Chrome, Firefox, Edge, Safari).
REQ-NFR-COMPAT-011	SAR Data Format Support	The system shall support common SAR data formats (e.g., Sentinel-1 GRD, SLC) for user-provided imagery and integrate seamlessly with data from sources like the Copernicus Browser.
REQ-NFR-PERF-012	Algorithm Inference Time	The system algorithm shall achieve an inference time of less than 40 milliseconds per image.
REQ-NFR-PERF-013	Scalability for Concurrent Users	The web-based interface shall handle a maximum of 5 concurrent users performing SAR analysis without significant degradation in response time.

Requirement ID	Title		Description
REQ-NFR-PERF-014	Data Ingestion Throughput		The system shall be capable of ingesting SAR data from Sentinel-1 at a rate of at least 10–1 GB/hour to keep up with incoming data volumes.
REQ-NFR-PERF-015	User Waiting Time		User queue takes less than 30 seconds per person waiting in line in front of the set user requesting for a scan of set area/provided image.
REQ-DARK-PERF-001	Detection Robustness		The system shall maintain high detection accuracy for dark vessels under various environmental and operational conditions, including different sea states and lighting conditions.
REQ-DARK-PERF-002	Integration Latency		The system shall process and integrate SAR and AIS data with minimal delay to enable near-real-time dark vessel detection and tracking.
REQ-DARK-DATA-003	Historical Utilization	Data	The system shall efficiently manage and utilize large volumes of historical AIS data for training and validation of dark vessel detection algorithms.
REQ-DARK-SEC-004	Data Privacy and Security		The system shall ensure secure handling and storage of AIS and SAR data, complying with applicable data protection regulations and policies.
REQ-DARK-EXT-005	Multi-Source Support	Data	The system shall be extensible to incorporate additional datasets (e.g., RF signals, optical imagery) in future versions to enhance dark vessel detection capabilities.
REQ-DARK-UI-006	Visualizations		The system interface shall visually distinguish between AIS-confirmed vessels and detected dark vessels with appropriate confidence indicators.
REQ-DARK-RELIABILITY-007	False Alarm Rate		The system shall keep the false alarm rate for dark vessel detection below 0.2% in open ocean environments by employing multi-source validation techniques and advanced filtering.

4 System Design

The SAR Ship Detection System employs a modular deep learning pipeline that processes and analyzes synthetic aperture radar (SAR) imagery for precise ship and dark vessel detection. Its architecture integrates multi-source SAR data acquisition, robust preprocessing (band selection, denoising, land masking), and advanced ML inference (YOLOv11m)—all optimized for both edge (Jetson AGX Orin) and cloud deployments. The post-detection workflow fuses SAR results with AIS data to flag non-cooperative ships and suspicious behaviors, substantially enhancing maritime surveillance against threats like illegal fishing and trafficking. The system delivers actionable outputs via intuitive interfaces for site selection, image upload, and map-based monitoring, supporting real-time, scalable use in operational environments.

4.1 High-Level System Architecture

The proposed workflow outlines a comprehensive pipeline for synthetic aperture radar (SAR) ship detection using deep learning techniques. The process begins with a literature review covering SAR sensor specifications, preprocessing methodologies, and benchmarking of existing public datasets and detection architectures. Based on this review, a data collection phase is initiated by selecting and merging the most relevant open-source SAR datasets. To address limitations in existing datasets particularly the absence of critical polarizations such as VV polarization, which is highly sensitive to metallic surfaces additional SAR scenes are incorporated to enrich the dataset and enhance detection capabilities.

The next stage involves data preprocessing and augmentation to improve model robustness under diverse maritime conditions. Following this, models are trained on the curated dataset and evaluated through a rigorous testing and comparison study. If performance metrics (e.g., mAP, recall, precision) are unsatisfactory, the workflow iterates back to refine preprocessing or augmentation strategies.

In the post-detection phase, the results obtained from the SAR-based ship detection model are systematically compared against Automatic Identification System (AIS) data to identify and flag potential dark vessels, ships that are not broadcasting AIS signals or whose AIS data does not correspond to detected radar signatures. This critical step enables the identification of vessels exhibiting suspicious behavior or engaging in activities such as illegal fishing, smuggling, or unauthorized transshipment, thereby enhancing maritime domain awareness.

Once the model achieves acceptable performance, it undergoes optimization and deployment. The final implementation supports two application domains: (1) an interactive web dashboard designed as a lightweight ecosystem for end-users, offering intuitive usage, interactive reporting capabilities, and the ability to visualize and analyze flagged dark vessel detections; and (2) an embedded deployment on the NVIDIA Jetson AGX Orin platform, enabling real-time inference and prediction directly on raw, full-frame SAR imagery without requiring prior land-ocean segmentation, and supporting onboard comparison with AIS feeds for dark vessel identification.

This dual-targeted architecture ensures both operational deployment in edge environments and user-friendly analysis for maritime monitoring and decision-making.

4.2 Detailed Component Design

4.2.1 Data Acquisition and Management

The data acquisition algorithm for the SAR ship detection project involves a systematic selection and integration of publicly available datasets from diverse sources, including Kaggle, GitHub, and Roboflow, to construct a robust training dataset.

Initially, a screening process identified two prominent public datasets: the SAR-Ship Dataset, comprising 43,819 images (10,2 Gaofen-3 and 10,8 Sentinel-1) with a resolution of 3–25 m, containing 59,535 ship instances of 256×256 pixels, and the SSDD (SAR Ship Detection Dataset), consisting of 1,160 images from Sentinel-1, RadarSat-2, and TerraSAR-X with 1–15 m resolution, featuring 2,358 ship instances of 500×500 pixels. These datasets were merged to form a unified corpus; however, a critical deficiency in VV polarization polarization highly effective for detecting metallic surfaces was identified. To address this, additional data were downloaded from Copernicus, and the SNAP software

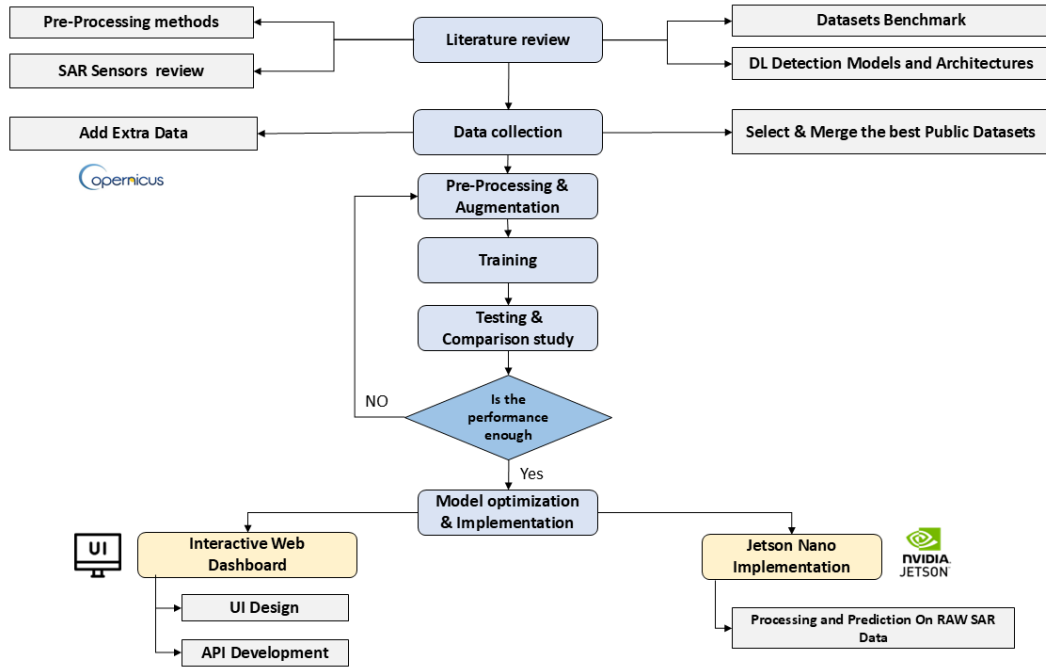


Figure 1: High Level System Architecture Flowchart

was employed to extract VV polarization bands, exporting them as PNG files.

Subsequently, the AIS API was utilized to enable automated labeling, enhancing dataset accuracy. Finally, these enriched data were integrated to yield a final dataset encompassing multiple bands and resolutions, optimized for advanced ship detection and tracking applications.

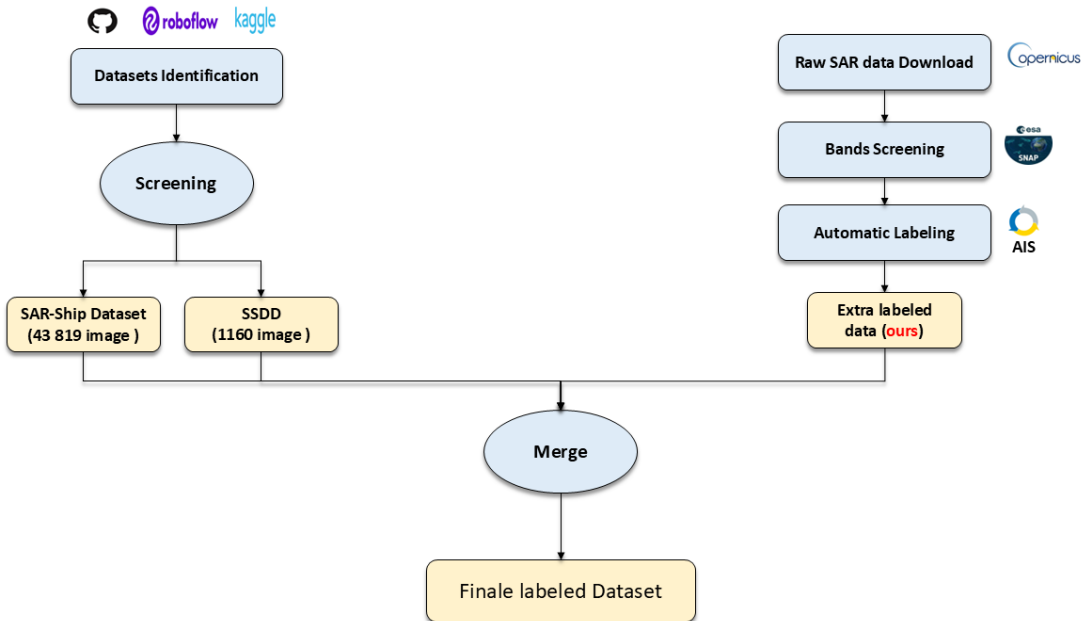


Figure 2: Data Acquisition Flowchart

4.2.2 Band Processing Subsystem

Ship detection using Sentinel-1 SAR data is challenged by the complexity of the maritime electromagnetic environment and the typically weak or concealed signatures of such vessels. Efficient band processing maximizes detection sensitivity, reduces false alarms, and highlights faint signatures, thus making each step vital.

The main purpose behind band processing is to transform raw SAR data into a form where subtle vessel signatures can be effectively isolated and enhanced. Different polarization bands (VV/VH) contain complementary information; by carefully selecting, filtering, and correcting these bands, we increase the probability of successful detection even under challenging conditions (low visibility, cluttered backgrounds, variable sea states). The following steps are outlined in the diagram represented in Fig. 3.

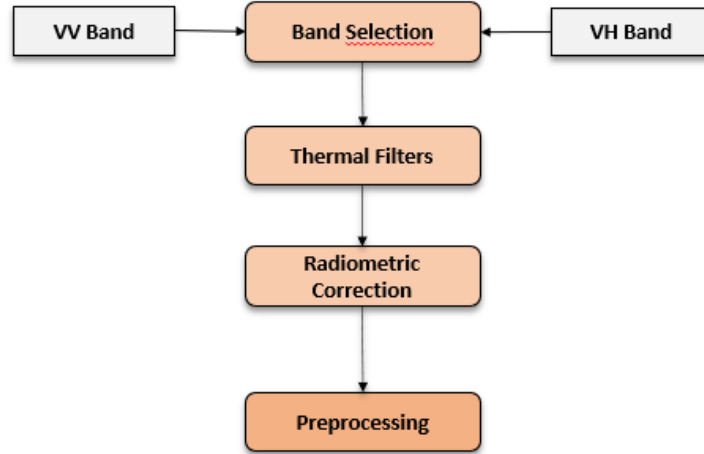


Figure 3: Band Processing Subsystem

Band selection Polarization refers to the orientation of the plane in which a transmitted electromagnetic wave oscillates. Radar can collect signals in different polarizations by controlling the analyzed polarization in both the transmit and receive paths. While the orientation can occur at any angle, SAR sensors typically transmit linearly polarized. The horizontal polarization is indicated by the letter H, and the vertical polarization is indicated by the letter V.

The advantage of radar sensors is that signal polarization can be precisely controlled on both transmit and receive. Signals emitted in vertical (V) and received in horizontal (H) polarization would be indicated VH polarization. Alternatively, a signal that was emitted in horizontal (H) and received in horizontal (H) would be indicated HH polarization, and so on. Examining the signal strength from these different polarizations carries information about the structure of the imaged surface based on the following types of scattering: rough surface, volume, and double bounce.

Sentinel-1 provides dual-polarization data (VV and VH) that encode different scattering mechanisms. VV polarization typically provides strong responses from metallic surfaces and calm water, while VH (cross-polarization) is particularly sensitive to volume scattering and can detect smaller vessels or those with complex geometries that might be missed in co-polarized channels. For dark vessel detection, the complementary nature of these polarizations is crucial, as vessels attempting to reduce their radar signature in one polarization may still be detectable in the other.

Thermal Filters Thermal noise in SAR imagery appears as random fluctuations that can mask weak vessel signatures, particularly in cross-polarization channels like VH where thermal noise is most prominent. For dark vessel detection, effective thermal filtering is critical because the target signatures may be only marginally above the noise floor. Thermal noise also creates interswath discontinuities that can generate false alarms in detection algorithms.

Radiometric correction Radiometric calibration is essential for converting raw digital numbers (DNs) to standardized backscatter coefficients that represent the true radar reflectivity of targets. This correction accounts for system gain variations, range dependencies, and acquisition geometry effects. For dark vessel detection, accurate radiometric calibration ensures that weak vessel signatures are not lost due to calibration artifacts and enables consistent detection thresholds across different acquisitions and geometric conditions.

4.2.3 Preprocessing Subsystem

Accurate ship detection in Synthetic Aperture Radar (SAR) imagery relies on a robust preprocessing pipeline designed to enhance critical ship features while suppressing noise, clutter, and irrelevant backgrounds. SAR images pose unique challenges such as speckle noise, low contrast under specific sea conditions, and interference from land areas, which this pipeline addresses (Figure 4).

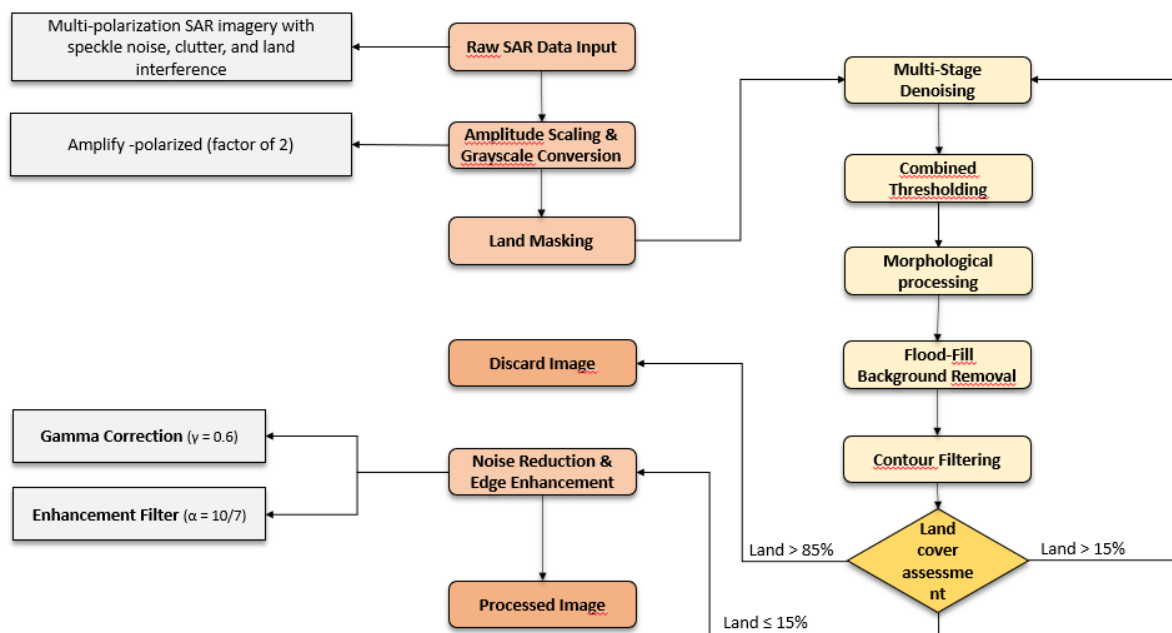


Figure 4: Preprocessing Subsystem Flowchart

The preprocessing workflow consists of three main stages: amplitude scaling with grayscale conversion, noise reduction and edge enhancement, and land masking.

Amplitude Scaling & Grayscale Conversion The first preprocessing step involves amplifying the VV-polarized radar returns by a factor of 2. This linear scaling brightens low-intensity backscatter values, improving the visibility of subtle features such as metallic ship structures against sea clutter while maintaining relative intensity relationships. The adjustment compensates for SAR's inherent dynamic range limitations, ensuring that dark, smooth sea surfaces remain distinguishable from bright, high-backscatter targets like ships. SAR imagery captures the backscattered radar signal, which is proportional to the target's reflectivity and surface roughness.

Linear amplitude scaling is a simple and effective method to improve visualization because it preserves relative contrast. Unlike nonlinear transformations, linear scaling maintains the proportionality between pixel values, which is crucial for target discrimination. It also adapts well to human vision, which perceives brightness logarithmically, making subtle backscatter variations in dark areas more perceptible without saturating bright targets.

This $2\times$ scaling enhances the interpretability of SAR imagery by accentuating ship signatures — metallic structures and complex geometries on ships produce stronger backscatter that becomes more pronounced after scaling — and suppressing noise in dark regions (e.g., smooth sea surfaces appear

brighter, reducing false negatives).

Following amplification, the adjusted VV channel is duplicated across all three RGB channels to produce a grayscale image. Grayscale rendering aligns with conventional SAR visualization practices, removing potential color bias and maximizing contrast for backscatter variations. Since ship detection often relies on textural and geometric differences, grayscale ensures optimal clarity for both human analysts and algorithms. The VV polarization channel is particularly sensitive to surface roughness and vertical structures, making it ideal for ship detection, as ships typically have stronger backscatter than the surrounding sea.

Unlike optical imagery, SAR data does not inherently contain spectral color information. False-color composites can introduce perceptual bias without adding meaningful features for single-channel (VV) ship detection. Grayscale preserves absolute intensity values crucial for distinguishing ships from sea clutter. For more detailed information, see Appendix A.1.

Noise Reduction and edge enhancing This preprocessing step aims to reduce noise in the SAR image and enhance the shapes of ships, minimizing the risk of false negatives by preventing the AI from confusing noise with actual targets. The process starts with a gamma correction filter using a gamma value of 0.6. Since ships appear very bright in VV-polarized SAR images, this gamma correction reduces overall image brightness, significantly dimming the noise—which is less bright—without obscuring the ships. This effectively makes the noise close to zero, while the ships remain visible.

After gamma correction, an image enhancement filter with an alpha of approximately 1.43 is applied. This filter boosts brighter regions more than dimmer ones, further enhancing ship shapes and diminishing any residual noise. The combined effect improves the visibility and definition of ships, aiding both human interpretation and algorithmic detection.

Gamma correction in SAR imagery adjusts contrast nonlinearly to enhance detail visibility and make features more distinguishable without altering average brightness. This enhances important image details such as ship edges, leading to better detection performance.

The processed result is illustrated in Figure 5. For more detailed information, see Appendix A.2

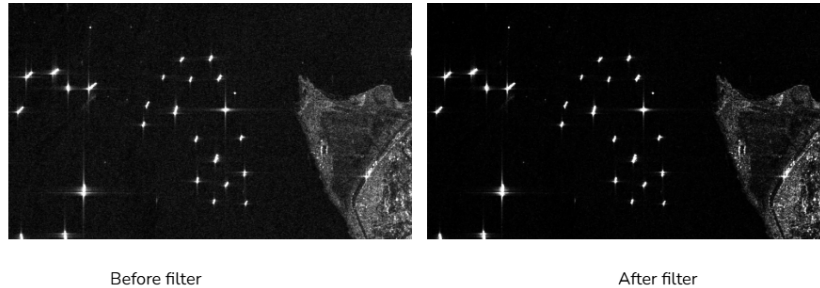


Figure 5: Before & After: Noise Reduction & Edge Enhancement

Land Masking The final step in the preprocessing pipeline for ship detection in SAR images is land segmentation, which aims to separate land areas from water bodies to reduce false positives in ship detection. The implemented approach consists of several stages, combining advanced image processing techniques to achieve robust land masking. In SAR-based land-water segmentation, the primary objective is robust separation of terrestrial and aquatic regions not the preservation of fine details such as ship signatures or wave textures. However, this task is complicated by speckle noise, an inherent granular distortion in SAR imagery caused by coherent radar wave interference. Unlike additive noise, speckle is multiplicative, meaning it scales with local backscatter intensity, obscuring true land-water boundaries. While conventional filters (e.g., Gaussian blur) indiscriminately smooth noise and edges alike, the Lee filter excels by adaptively suppressing speckle while preserving critical structural features like coastlines.

The Lee filter plays a vital role here, offering adaptive speckle suppression while preserving coastline features, essential for reliable maritime analysis. By producing an image optimized for threshold-based segmentation, water regions become more uniform and coastlines remain sharply defined, reducing false

positives and enabling effective downstream ship detection.

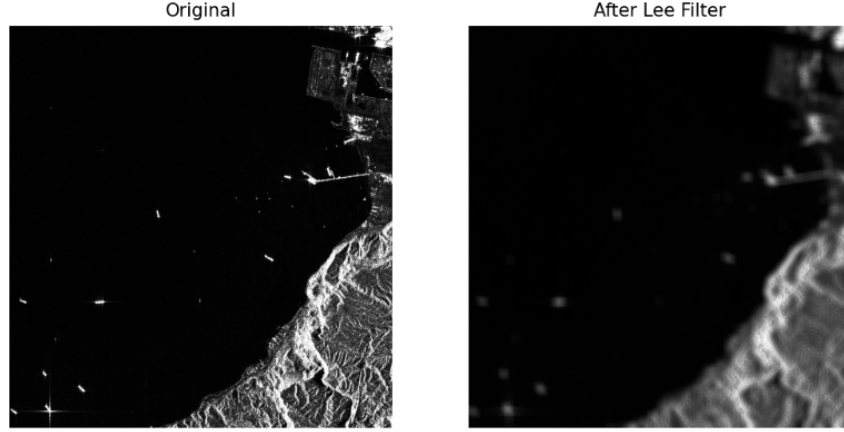


Figure 6: Before & After: Lee filter Application

A multi-stage denoising and enhancement strategy is applied before land segmentation: Gaussian Blurring reduces random noise without excessive edge loss, while CLAHE locally improves image contrast without amplifying noise—particularly critical for distinguishing land and water in regions with complex backscatter.

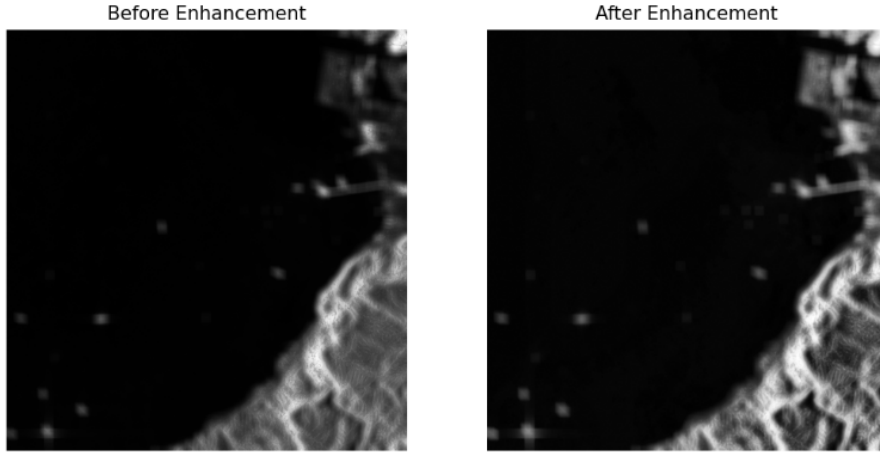


Figure 7: Before & After: Multi-stage Denoising

Thresholding then converts the denoised image into a binary mask, separating land and water. Since SAR images display significant local and global intensity variation, robust detection employs both Otsu’s global thresholding (for overall separation) and adaptive Gaussian thresholding (for local differences). The combination ensures robust land detection, improving model reliability in diverse scenes.

Morphological processing refines the binary mask, filling small holes and removing isolated noisy pixels. Flood-filling further improves segmentation by ensuring water regions are properly connected and labeled, minimizing fragmented false positives.

Contour filtering removes residual artifacts and insignificant small regions from the land mask using area-based criteria, allowing only the most relevant landmasses to remain.

Contour filtering removes residual artifacts and insignificant small regions from the land mask using area-based criteria, allowing only the most relevant landmasses to remain. To further improve accuracy, especially in complex coastal areas, the segmentation pipeline performs iterative refinement. By evaluating the percentage of detected land area and adaptively improving the segmentation, it automatically balances

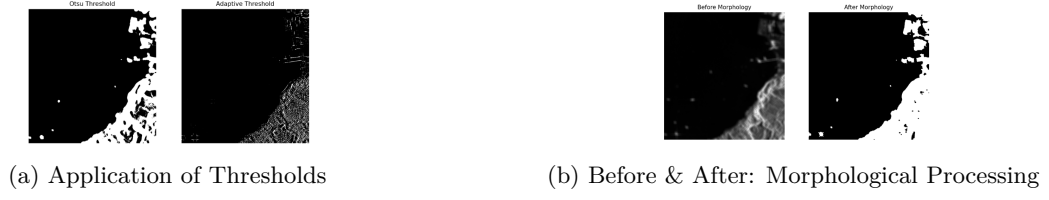


Figure 8: Comparison of image processing techniques

over- and under-segmentation, leading to robust land masking—critical for accurate ship and dark vessel detection.



Figure 9: Final Mask Obtained

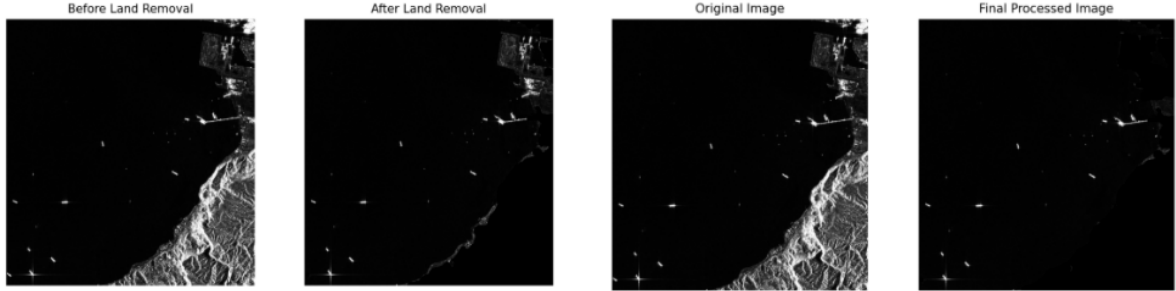


Figure 10: Comparization between Original Image and Results of first and second preprocessing iteration

4.2.4 Machine Learning Detection Model

The performance comparison graph provided by Ultralytics, which plots mean average precision (mAP) at IoU thresholds from 0.5 to 0.95 against latency in milliseconds per image on a T4 TensorRT FP16 platform (Figure 11), serves as a critical diagnostic tool for evaluating candidate models for our SAR-based ship detection and tracking project.

Analyzing the graph reveals that YOLOv11n achieves a modest 40% mAP with a latency of 2-3 milliseconds, indicating high speed but insufficient precision for detecting small ships in SAR imagery, where targets often occupy a tiny fraction of the image. YOLOv11l, while offering a higher mAP of 54% with 8 milliseconds latency, exceeds the computational capacity of the resource-constrained NVIDIA Jetson AGX Orin, a low-power edge device with a 128-core Maxwell GPU, making it impractical for

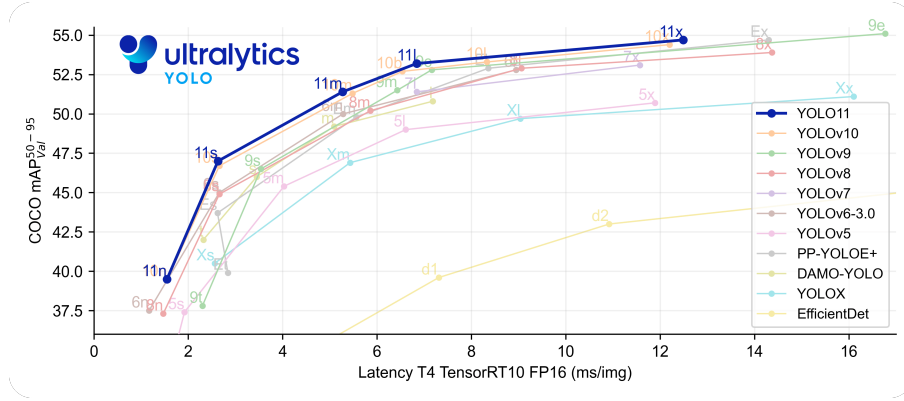


Figure 11: Performance Comparison Graph

real-time deployment. YOLOv8m and YOLOv10m, with mAP values of 46% and 50% respectively and latencies around 6 milliseconds, provide moderate performance but fall short in precision for small-object detection and remain borderline for Jetson Nano compatibility.

In contrast, YOLOv11m emerges as the optimal choice, delivering a balanced 52.5% mAP with a latency of 4-5 milliseconds, which aligns with the project's dual requirements of high precision for small ship detection and efficient operation on limited hardware. This model's architectural enhancements, likely including improved feature pyramid networks or anchor-free detection heads, enhance its ability to capture fine details in SAR images, as evidenced by its superior mAP compared to predecessors.

Furthermore, with optimizations reducing memory usage and accelerating inference up to 4x it can achieve real-time processing rates of 10-20 frames per second on the Jetson AGX Orin, as supported by studies like [20] on edge optimization of YOLO models and [21] on efficient neural networks, solidifying YOLOv11m as the best fit for our maritime surveillance application.

4.3 Data Flow and Workflow

This section provides a detailed, step-by-step explanation of the data pipeline within the SAR Ship Detection system, elaborating on the high-level architecture presented previously. It provides an overview of the solution implementation and describes the processes that each stage goes through from raw SAR data ingestion to the final output of detected and tracked ships.

The data flow within the SAR Ship Detection Project follows a structured pipeline designed for efficiency and accuracy. The primary stages are:

- **SAR Data Acquisition:**
 - **Process:** The pipeline initiates with the acquisition of raw Synthetic Aperture Radar (SAR) imagery. Acquisition of data is done via the Copernicus Browser. For the hardware implementation (Jetson Nano Edge) various raw SAR image files are preloaded into the file system that it randomly chooses from upon use. For the cloud implementation the SAR data is either uploaded from the user or queried from Copernicus via user requests initiated through Google Earth.
 - **Details:** Data can be downloaded directly from Copernicus in either raw or processed formats. Data can be in multiple file types such as .png and .jpeg as well as .tiff format which allows for use with larger image sizes through special software. The Data is uploaded to the solution via the BlueGuard website which is running the model. Data uploaded by users is first fed into a queue to ensure the page can handle the traffic.
- **Initial Data Ingestion and Storage:**
 - **Process:** Upon receiving a user request via the Streamlit web interface, the system initiates a data ingestion process that either: (1) Downloads SAR imagery from the Copernicus API based on user-specified coordinates and time range. (2) Accepts user-uploaded SAR .tiff, .png,

or .jpeg images directly through the web interface. (3) Finally, optionally queries AIS data for the corresponding area and time from a third-party API. Once retrieved or uploaded, the data is temporarily stored for processing and version tracking.

- **Details:** Data is stored in local server only while in use. A structured directory or database system is utilized to manage large volumes of SAR images. The images once outputted to the user will be automatically deleted from the server once the unique session is over or times out. SAR images and associated metadata are stored locally in a structured directory on the host laptop. Each request is isolated in a temporary directory, identified by a unique session or request ID, which avoids filename collisions and supports simple cleanup. Due to limited disk space on the host laptop, data is stored ephemeraly. A background cleanup job (using a lightweight Python scheduler like APScheduler) deletes session data older than a set threshold (e.g., 15 minutes or after download).
- Each ingestion is isolated to its own session. Streamlit’s session-based design supports this naturally, but in addition, image download requests are asynchronous using threads or background tasks to avoid freezing the Streamlit UI.

- **Preprocessing Queue Management:**

- **Process:** Once a SAR image is ingested, it is placed in a preprocessing queue. This queue serializes and throttles incoming processing requests, ensuring that system memory and CPU/GPU resources are not overloaded when many users access the site concurrently.
- **Details:** A simple in-memory First-In-First-Out (FIFO) job queue is implemented using Python. A dedicated background worker thread or asyncio task continuously monitors the queue and processes jobs one by one. While in the queue, users are shown an estimate of their position in the queue and approximate wait time. This is implemented with Streamlit’s session state and periodic UI refresh (st.experimental_rerun or st.autorefresh).

- **SAR Image Preprocessing (Subsystem):**

- **Process:** The raw SAR images undergo a series of critical transformations to enhance features, reduce noise, and prepare them for machine learning inference. This stage is crucial for improving detection accuracy.
- **Details:**

- * **Polarimetric Band Selection**

Technique: VV polarization Polarization Band Selection

Description: The VV polarization band is chosen due to its sensitivity to sea surface roughness and ship wake visibility, making it ideal for ship detection.

- * **Amplitude Scaling and Grayscale Conversion**

Technique: Linear Intensity Amplification and RGB Channel Duplication

Description: The VV polarization channel is amplified by a factor of 2 to improve contrast between ships and sea clutter, then converted to grayscale by copying it into all three RGB channels.

- * **Noise Reduction and Edge Enhancement**

Technique: Gamma Filtering and Alpha-Based Enhancement

Description: A gamma correction ($\gamma = 0.6$) suppresses dim background noise while retaining ship features, followed by an enhancement filter ($\alpha = 10/7$) to increase the brightness of ships and suppress residual noise.

- * **Land Masking:**

Techniques: Lee Filter + CLAHE + Dual Thresholding + Morphological Processing + Flood-Fill + Contour Filtering + Iterative Refinement

Descriptions:

- Lee Filter: Adaptive speckle reduction filter preserves coastline boundaries while smoothing sea regions.

- CLAHE: Enhances local contrast within the image while preventing over-amplification of noise. - Gaussian Blurring: Reduces high-frequency noise while preserving structure using a Gaussian convolution kernel. - Otsu’s Thresholding: Global thresholding to segment land and sea by minimizing intra-class intensity variance. - Adaptive Gaussian Thresholding: Locally adaptive binarization to handle spatial variations in intensity. - Morphological

Processing: Closing and opening operations correct segmentation defects (fill gaps, remove noise). - Flood-Filling: Segments water from land by propagating from known water regions (image corners), then inverts to isolate land. - Contour Filtering: Removes small false-positive land segments by filtering based on area threshold. - Iterative Refinement: Repeats segmentation and filtering until land coverage is within valid thresholds; terminates or refines further based on percentage rules (e.g., >85% land coverage aborts detection).

* **Format Conversion and Standardization:**

Technique: Conversion to Numpy Array and Normalization

Description: The preprocessed grayscale image is converted into a normalized Numpy array with pixel values scaled between 0 and 1 for direct input to the deep learning model.

• **Ship Detection Inference (Machine Learning Model):**

- **Process:** The preprocessed SAR image is fed into the trained ship detection deep learning model. This model analyzes the image to identify potential ship instances. The prediction returns the annotated image and JSON file containing information regarding the ship geographical coordinates and confidence etc.
- **Details:** The preliminary trained model is the YOLOv11m for its good balance of precision and latency. This model was used for preliminary analysis of the overall solution and for development of the end-to-end system. The models are trained via PyTorch with the respective datasets (preliminary vs final) and the weights are converted to ONNX format from which can be used for inference in any format, specifically TensorRT, which is used for the Jetson Nano architecture. Predictions of ships are not classified into distinct ship types, the model is only trained to identify whether or not there is a ship within the data. The model is deployed on the Jetson Nano and locally on the demo website host.

• **Output Generation:**

- **Process:** The final results, including detected ship bounding boxes, confidence scores, and unique tracking IDs, are compiled and formatted for output.
- **Details:** This output can be presented in various forms depending on the user interface:
 - * **Structured Data Output:** Generation of machine-readable files (JSON) containing all relevant detection and tracking metadata (coordinates, timestamps, IDs).
 - * **Annotated Image:** Overlaying bounding boxes, labels, and tracking IDs directly onto the original or preprocessed SAR imagery for visual inspection. The annotated image can be downloaded as a whole, or individual identified ships can be downloaded separately can be downloaded individually.
 - * **Data Downlink/Transmission:** For edge deployments (Jetson Nano), relevant output data is prepared for transmission to ground stations or other connected systems.

• **User Interface / Visualization:**

- **Process:** The processed results are delivered to the user through one of the defined interfaces, allowing for interaction and visualization of the detected and tracked ships. The holistic interface will be made using Streamlit and hosted as a webpage containing three pages for each sub-interface type listed below. These are different ways users can request or view use of the solution and models depending on their desired input type.
- **Details:**
 - * **Jetson Nano Interface:** Processed data is “downlinked” and visualized on a local display or streamed to a ground control application, potentially integrating with on-ground AIS data. Statistical insights are also provided. The user cannot request specific data, the jetson nano will come pre-loaded with random SAR image files, its functionality may be expanded once integrated into a satellite.
 - * **Web-based Map Interface:** Users interact with a Google Earth-style map to select a region of interest. This region of interest will be limited to a maximum size and square shape, this is to reduce the load on host machine. An API fetches the latest SAR data, processes it and sends it to the model for prediction. The result goes through the statistical insights to be displayed to the user.

- * **Image Upload Interface:** Users upload their raw SAR images. the system processes the image and sends it to the model for prediction. The result goes through the statistical insights to be displayed to the user.

5 Implementation and Development

This section details the software and hardware setup underpinning the SAR Ship Detection Project, highlighting the integration of high-performance edge devices (Jetson AGX Orin), cloud tools (Google Colab, Roboflow), collaborative coding environments (GitHub), and streamlined interfaces (Streamlit). Robust implementation leverages a modular pipeline for image acquisition, annotation, model training, and deployment, supported by efficient dependency management (Docker) and virtualization. Edge-specific algorithms (tile-based inference, advanced pre- and post-processing) are optimized for real-time maritime surveillance, ensuring high detection accuracy even in challenging environments. The section provides guidance for both local and edge deployments, tools for performance monitoring, and methodologies for converting model weights for accelerated inference. These elements collectively facilitate a scalable, reproducible, and operational ship and dark vessel detection system.

5.1 Software and Hardware Environment Setup

5.1.1 Training Setup, Hardware + Software

- **Roboflow** Roboflow is a powerful, end-to-end computer vision platform that simplifies everything from dataset management and image annotation to model training and deployment. It supports over 40 data formats, offers AI-assisted labeling, and enables training with advanced architectures like YOLO. We used Roboflow to easily upload and annotate our dataset, then successfully trained a sample YOLO model within its intuitive interface.
- **Google colab** A versatile, cloud-based platform for writing and executing Python code in a web browser, requiring no setup. Built on Jupyter Notebook, it's a go-to tool for machine learning, data analysis, and education, offering access to powerful GPUs and TPUs. We leveraged Colab for various data processing tasks and to efficiently train our model, evaluating its suitability for our project in a streamlined, collaborative environment.
- **NVIDIA Jetson AGX Orin** A powerful, high-performance edge AI device delivering exceptional performance for on-orbit inference and data processing in demanding space environments. Featuring a 12-core ARM Cortex-A78AE CPU, 2048-core NVIDIA Ampere architecture GPU with 64 Tensor cores, and up to 64 GB LPDDR5 RAM with 204.8 GB/s memory bandwidth, it provides up to 275 TOPS of AI performance. This represents an 8X performance increase over previous generation modules while maintaining configurable power consumption between 15W and 60W. Compatible with comprehensive AI frameworks including TensorFlow and powered by NVIDIA JetPack SDK, the Jetson AGX Orin enables sophisticated onboard deep learning tasks such as object detection, segmentation, and multi-sensor fusion for dark vessel detection. Its advanced parallel processing capabilities and high-speed I/O support make it ideal for autonomous, AI-driven space applications requiring real-time analysis while significantly reducing raw data transmission requirements and optimizing bandwidth efficiency in resource-constrained orbital environments.
- **GitHub**: A web-based platform for version control and collaborative software development, built around Git. We utilized GitHub to host our project's codebase, facilitating seamless collaboration among team members and ensuring proper version control throughout the development lifecycle.
- **STREAMLIT**: an open-source Python library that simplifies the creation of custom web applications for machine learning and data science. We employed Streamlit to develop an intuitive user interface for our project, allowing for easy interaction with our trained models and visualization of results.

5.1.2 Implementation Hardware/Edge

The primary hardware utilized for edge deployment and development in the SAR Ship Detection Project is the NVIDIA Jetson AGX Orin. Its specifications are detailed as follows:

- Model: NVIDIA Jetson AGX Orin Developer Kit (64GB version)
- GPU: NVIDIA Ampere architecture with 2048 NVIDIA CUDA cores and 64 Tensor Cores
- CPU: 12-core ARM Cortex-A78AE v8.2 64-bit CPU @ 2.2 GHz with 3MB L2 + 6MB L3 cache

- Memory (RAM): 64 GB 256-bit LPDDR5 @ 6400MHz (204.8 GB/s bandwidth)
- Storage: 64GB eMMC 5.1
- Power Input: Configurable between 15W and 60W for optimal performance and energy efficiency
- AI Performance: Up to 275 TOPS of AI performance

5.1.3 Operating System and Core Software Versions

- Ubuntu: 22.04 LTS
- Python version: 3.10
- NVIDIA JetPack: 6.2
- CUDA: 12.6
- cuDNN: 9.3
- TensorRT: 10.3

5.1.4 Dependency Management and Virtual Environment Setup

The dependency management and development and deployment environment are handled by docker compose containers. Within the root folder is the requirements.txt that is used by the web app container to install the required dependencies. Alternatively using the command `pip install -r requirements.txt` would also set up the environment if developing within .venv or adjacent environment.

As the dependency management is handled by docker, the requirements.txt has a loose minimum version package declaration which includes the main packages and lets pip handle the additional dependencies (which it does automatically).

The docker containers are the recommend virtual environment setup to use. The docker containers are created with docker compose which specifies the service names and additional information, notably that the jetson docker is intended for integrated GPU usage.

The two dockers services are web_app and jetson_app where the web app service contains the entirely repository excluding the jetson container and the .dockerignore files of course. Whereas the jetson container has completely independent of the rest of the repository and only contains the Jetson AGX Orin folder and has a different image and packages which are optimized specifically for CUDA support for use in the NVIDEA Jetson AGX Orin model.

5.1.5 Accessing the Jetson Nano

To ssh to the Jetson Nano use:

```
ssh jetson@<nano_ip_address>
```

5.1.6 Essential Command Line Commands

- **jtop**: (After `sudo pip3 install -U jetson-stats` and reboot) A fantastic utility to monitor CPU, GPU, memory, power, and temperatures on your Jetson Nano. Essential for optimizing performance. Type `q` to quit `jtop`.
- **sudo jetson_clocks**: Maximizes CPU/GPU/memory clocks for best performance. Useful before running demanding tasks. The `jtop` tool can also do this.
- **sudo reboot**: Restarts the Nano.
- **sudo shutdown now**: Powers off the Nano.

5.1.7 RAW Data prediction algorithm (Inference Slicer Method)

The designed function (Fig. 12) constitutes a modular inference framework for high-resolution remote sensing imagery, with a specific focus on maritime detection under noisy acquisition conditions. The algorithm integrates three principal stages: preprocessing, tiled inference, and post-processing with spatio-contextual filtering. In the preprocessing stage, the input image whether in raw TIFF or other formats is standardized through format conversion, grayscale transformation, and iterative denoising to suppress radar-induced artifacts and enhance object boundaries.

A binary land–water segmentation mask is computed to spatially constrain the search domain, thereby mitigating false positives arising from land-based structures. During the inference stage, the preprocessed image is partitioned into tiles of fixed dimension, ensuring tractable input sizes for the local detection model while preserving spatial detail. Each tile is independently processed by a convolutional object detector, producing a set of candidate bounding boxes characterized by centroid coordinates, width–height dimensions, and confidence scores. These detections are then subject to rigorous post-processing:

1. coordinate transformation converts relative bounding-box locations into absolute pixel positions;
2. area-based filtering discards abnormal detections with pixel surface above 2000, thereby eliminating noise-induced outliers and
3. land-mask intersection tests exclude objects overlapping terrestrial regions.

Validated detections are then accumulated, annotated on the denoised image, and cropped with margin expansion to generate auxiliary image patches for secondary inspection. For each confirmed detection, the pixel-based surface area is transformed into real-world units via the sensor’s ground resolution, and, when georeferenced data are available, pixel coordinates are mapped into geographic coordinates using a pixel-to-longitude/latitude conversion function.

Finally, the algorithm outputs four synchronized products: an annotated image containing all valid detections, a set of cropped ship patches, a global counter of detected instances, and a structured metadata file storing spatial, geometric, and geolocation attributes of each detection. By jointly integrating denoising, land–water masking, tiling-based inference, and multi-criteria filtering, the framework provides a robust and extensible pipeline for maritime surveillance, supporting both operational monitoring and large-scale geospatial analytics. The detailed pseudocode of this algorithm can be found in the appendix B for further reference.

5.1.8 Google Earth Engine Prediction API

The proposed workflow implements an end-to-end pipeline for maritime object detection that couples cloud-hosted data access with robust image conditioning and scalable, tile-based inference (Fig. 13). Input imagery is acquired from Google Earth Engine (GEE) for a user-defined footprint and time window and aggregated into a temporally stabilized composite to reduce transient noise.

The composite is then normalized by stretching its contrast between low and high percentile values and processed with a domain-adapted noise-reduction operator that preserves edges and small targets. A land/water mask is computed to restrict detection to the marine domain. The conditioned raster is subdivided into fixed-size tiles; each tile is analyzed independently by a trained detector optimized for small-object retrieval in noisy radar data. Candidate detections are converted to scene coordinates, scored, and filtered by simple plausibility tests — for example size-based thresholds and overlap checks with the land mask — to remove false alarms from clutter and shoreline features. For every validated detection the pipeline produces a compact cropped image with contextual margin, an annotation on the full-scene visualization, and a metadata record that contains pixel geometry, a metric area estimate derived from the scene resolution, and geographic coordinates when available.

Final products are delivered through an interactive interface that displays the annotated scene, summary statistics and charts, and an archive of single-instance extracts for manual review. The detailed pseudocode of this algorithm can be found in the appendix C for further reference.

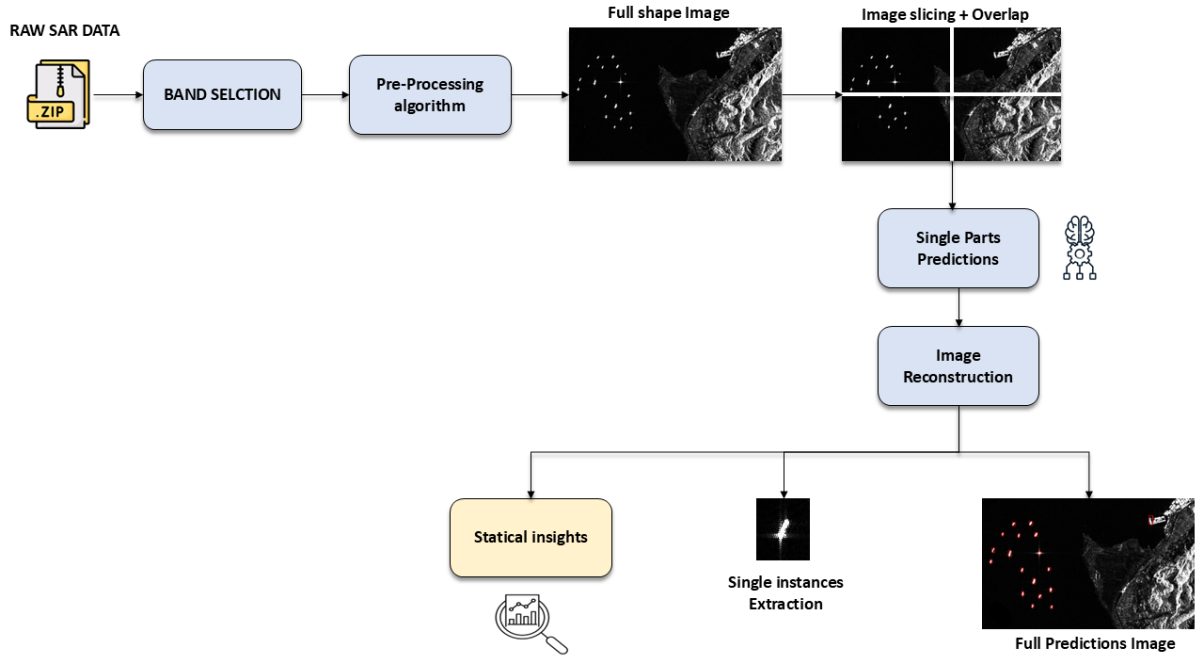


Figure 12: RAW Data Prediction Algorithm Flowchart

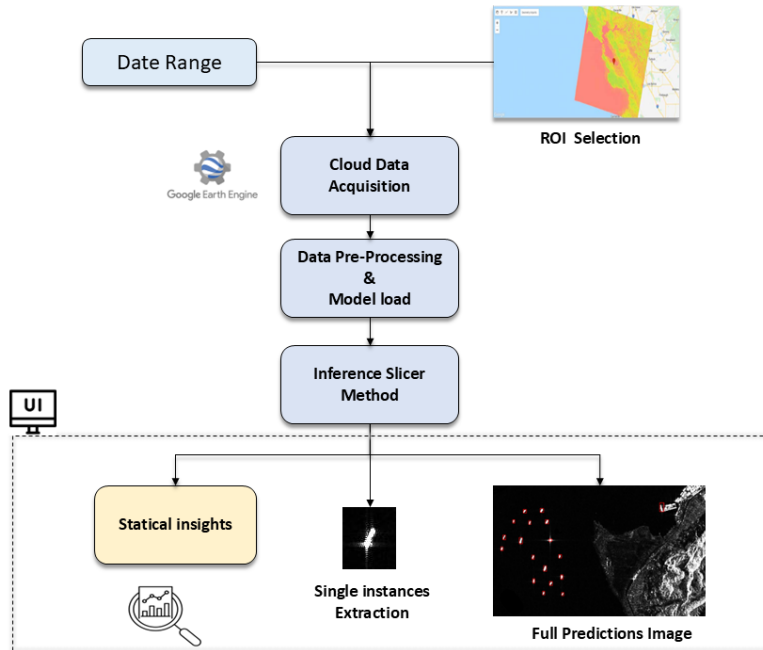


Figure 13: Google Earth Engine Prediction API Flowchart

5.1.9 AIS Integration

The AIS matching algorithm provides a structured methodology to associate vessels detected in satellite SAR imagery with entries from the Automatic Identification System (AIS), facilitating maritime monitoring and surveillance (see Fig 14)

For each detected ship, the algorithm first extracts the estimated geographic coordinates and detection timestamp, establishing a defined temporal window around the event.

The AIS dataset is then processed in manageable chunks, which allows efficient handling of large-scale data, and initial filtering removes invalid or incomplete records to maintain data quality. Each valid AIS record is evaluated, combining spatial distance to the detected ship and temporal offset from the

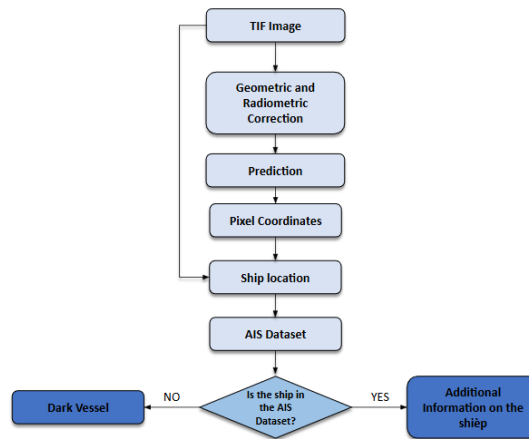


Figure 14: Enter Caption

detection time into a weighted matching score, retaining only the best-scoring candidate for each MMSI within the search radius.

Following evaluation of all possible matches, the algorithm enriches the detection metadata with AIS information, including positional coordinates, MMSI, timestamp, and the computed distance, time difference, and matching score. The final output is compiled into a structured JSON file, allowing seamless integration into analytical workflows.

Critically, if no AIS record satisfies the selection criteria for a given detection, the algorithm classifies the vessel as a “dark ship,” reflecting its absence from global AIS registries and highlighting potential unmonitored maritime activity.

Key operational steps (see Appendix ??) :

- Extract geolocation and timestamp for each detected ship.
- Define a temporal search window around the detection time.
- Process the AIS dataset in chunks, filtering out invalid or incomplete entries.
- Combine spatial and temporal metrics into a weighted score, retaining only the best candidate per MMSI.
- Enrich matched metadata and append results to a structured JSON output.
- Flag any unmatched ships as “dark ships” for further investigation.

5.2 Installation and Setup Guide

Web App Setup

1. Firstly, clone the repository SAR-SHIP-DETECTION.
2. Run:

```
pip install docker-compose
```

3. While installing, create a config.json file in the root folder. The format is provided in the CLAUDE.md markdown file to setup the Google Earth Engine APIs. You can leave the empty if the Google Earth Engine won't be used.
4. Run:

```
docker compose up web_app <optional flags: -d for detached mode>
```


5. Navigate in a browser to the corresponding localhost port, by default it is localhost:8501, if the command was run in detached mode it will not show the port however it is likely set to default.
6. To shut down the docker:

```
docker compose down
```

Jetson App Setup

1. To SSH to the Jetson use:

```
ssh jetson@<agx_ip_address>
```

2. A new project folder for the Jetson can be made with the `mkdir <directory_name>` command.
3. In the Jetson, clone the repository SAR-SHIP-DETECTION. Files outside of the Jetson AGX Orin folder can be safely deleted on the Jetson. Alternatively, if the repository is already located on a local device, the Jetson AGX Orin folder can be copied to the Jetson using the secure copy (`scp`) command.
4. If the Jetson AGX Orin is properly setup using the SDK, docker should already be installed. If not, refer to Section 5.3 Configuration.
5. If the ONNX file is not contained with the repository or different weights will be used, run the following command to convert from ONNX to TensorRT. If the model weights are not in ONNX format this can be done with the `onnxtransform.py` function.
6. Run:

```
trtexec --onnx = best .onnx -- save Engine = best_fp 16 .  
engine --fp16
```

7. To execute python code from the Jetson you can directly use the docker container. Within the folder are a `Final_function.py` and `inference_with_ONNX.py` which will take a test image from the inputs folder and create the results in the output folder. `Final_functions.py` includes pre-processing steps while the other does not. To run these files use the command:

```
docker compose up jetson_app <optional flags: -d for  
detached mode>
```

```
docker compose exec jetson_app python3 <filename.py>
```

For inference to the Jetson AGX Orin use ssh to connect to the terminal. On windows a program like PuTTY may have to be used, though WLS Ubuntu is recommended.

An alternative to directly writing onto the Jetson AGX Orin is to secure copy (`scp`) the project. For development create a virtual python environment and install the required packages.

The Jetson should already come equipped with TensorRT among other packages with the Jetson AGX Orin SDK. Format the model from ONNX format to TensorRT. Finally, run either the `inference_with_ONNX.py` or `Final_functions.py`. `Final_functions` includes the pre-processing steps for the data while the `inference_with_ONNX` only runs the detection. Important to note that the files within the input folder are assumed to already be radiometrically calibrated and have an applied orbital file.

5.3 Configuration

The `config.json` file is included in the `.gitignore`, thus as per the setup steps you need to create a `config.json` on a fresh install, mainly for use of the ground segment (`web_app`). It will throw an error at the start of the program without the file, while the file itself may be empty if the only desired usecase is the jetson functionality. The template for the config file can be found in the `CLAUDE.md` file in the root folder or in Appendix E.

Alternatively you can create a `.env` file and load API keys there, however it is recommended to use the `config.json` as the `.env` file compatibility might not be fully integrated everywhere in the repo.

The AIS Data Ingestion location is from NOAA 2024 database by default. The AIS Data is automatically downloaded into the `Ais_data` folder, where it will stay until it hits a certain file size buffer, by default 4.5 GBs. To change this value Then the file is popped and deleted. All the downloaded AIS data is stored as zip files, which are only extracted while in use and not stored on the device.

There is not support for integrating other AIS data source locations within the context of this project and the AIS is not integrated to any process from after the jetson inferences and “downlinks the data”, this means when changing the AIS provider will require some coding changes. This can be done in the `FullApp/functions.py` file. There is currently no framework for AIS API implementation already setup for use.

The webapp also outputs `.json` files `ship_metadata` and `ship_metadata_ui` after inference for the first time. These outputs can be found in the output folder in the `FullApp` folder, where as for the jetson there is also a dedicated input and output folder with different types of files as well as subimages. Nothing is done afterwards with the output images in the jetson container. At this point as explained in the future work section can have a configuration which allows to choose what gets downlinked and how much to reduce the downlink size subsequently what data would be excluded.

6 Model Training and Optimization

This section details the comprehensive process of preparing high-quality SAR ship detection datasets, training the YOLOv11m model using transfer learning, and optimizing the model for efficient deployment on the NVIDIA Jetson AGX Orin edge platform. Dataset curation involves merging diverse public SAR datasets, validating annotations with AIS and optical imagery, and partitioning data for robust model evaluation. Training utilizes cloud GPU resources with carefully tuned hyperparameters to maximize detection precision. The optimization phase converts the model to the ONNX format followed by aggressive simplification and deployment using TensorRT, leveraging Ampere GPU architectures and dual DLAs on the AGX Orin to achieve real-time inference, low latency, and power-efficient operation, while maintaining high accuracy critical for maritime and dark vessel surveillance applications.

6.1 Dataset Preparation Review

After conducting preliminary research on public datasets containing Synthetic Aperture Radar (SAR) ship imagery, a shortlist of high-quality candidates was compiled. Upon closer evaluation, two datasets were selected: the SAR SSDD Dataset and the SAR Ship Dataset. These datasets were merged to create a preliminary dataset, which was then used to train a minimum viable product (MVP) model.

Roboflow was used to upload and manage the datasets. Using the platform, the datasets were merged and split into training, validation, and test subsets. Roboflow also converted the data to the YOLOv11 format and resized the images to 640 by 640 pixels, the recommended input size for models in the YOLO family. No augmentation was applied to the preliminary dataset.

The final dataset builds upon the preliminary version by incorporating slightly lower-resolution public SAR data and edge cases such as vessels in glacial waters or near oil rigs. These additions enhance the model's robustness and help reduce false positives.

The final dataset was annotated using Roboflow, with ship positions verified against Automatic Identification System (AIS) data corresponding to the SAR data acquisition period. Where available, optical imagery from similar timeframes was also used for verification. This validation process reduces the likelihood of human error during annotation, thereby minimizing the introduction of noise. Roboflow ensures high-precision bounding boxes, contributing to the overall quality of the annotated dataset.

The final dataset was then merged with the preliminary dataset using Roboflow. It was partitioned into training, validation, and test subsets in a 70%, 20%, and 10% split, respectively.

To export the dataset for training in Google Colab or a similar environment, Roboflow provides three methods: direct download as a ZIP archive, cURL via the command-line interface, or use of the Roboflow Python package.

To install and access the dataset using the Roboflow package, the following commands can be used:

```
!pip install roboflow
from roboflow import Roboflow

rf = Roboflow(api_key="API_KEY_HERE")
project = rf.workspace("WORKSPACE_NAME").project("PROJECT_NAME")
version = project.version(1)
dataset = version.download("EXPORT_FORMAT")
```

Roboflow also offers the option to automatically generate a Google Colab notebook pre-configured with the appropriate installation and dataset download commands. Users can initiate these commands by clicking the run button next to each code block. While Roboflow typically autofills dataset details, users should ensure that all required information is correctly specified.

6.2 Training Process

The YOLOv11m model for SAR ship detection was trained using a transfer learning strategy. We initialized the model with pre-trained weights from the COCO dataset, allowing the network to adapt

more efficiently to the SAR image domain. This approach significantly reduced training time and improved convergence stability.

Training was conducted on a Kaggle Notebook with access to a Tesla T4 GPU (16GB), running in a cloud-hosted environment with PyTorch, CUDA 11.x, and Python 3.10. This setup provided sufficient resources for training the medium-sized YOLOv11m model on 640×640 SAR images.

Hyper-parameters used:

- Model architecture: 231 layers
- Parameters: 20,053,779
- GFLOPs of computation: 68.2
- Learning rate: 0,002
- Momentum: 0.9
- Epochs: 100
- Image Size: 640×640 (for both training and validation)

Progress was tracked using TensorBoard and Ultralytics' built-in training logger, reporting training/validation loss, precision, recall, mAP@0.5, and mAP@0.5:0.95. Visualizations of label distributions were saved automatically.

To train the YOLOv11m model for SAR ship detection on Kaggle, follow these steps:

1. Set Up the Kaggle Notebook Environment:

- Create a new Kaggle NotebookEnsure your Kaggle notebook has GPU enabled
- Install Ultralytics' YOLOv11 framework

```
!pip install ultralytics --upgrade2
```

2. Prepare the Dataset:

- Upload or mount your SAR ship detection dataset in YOLO format

```
/images/train, /images/val, /labels/  
train, /labels/val
```

- Create a dataset config YAML file, e.g.

```
data/sar_ship.\acrshort{yaml}:
```

3. Run the Training Command

```
!yolo detect train --model \acrshort{yolov11m}.\  
acrshort{yaml} --data data/sar_ship.\acrshort{\  
yaml} \  
--epochs 20 --img 640 \  
--weights yolov11n.pt --device 0 \  
--project runs/train --name yolov11m_sar_ship \  
--workers 2
```

6.3 Optimization techniques for Jetson AGX Orin

To ensure optimal performance in constrained environments characterized by low latency, embedded deployment, and strict power efficiency, a rigorous optimization phase was conducted for the AI models deployed on the NVIDIA Jetson AGX Orin platform. The AGX Orin, delivering up to 275 TOPS of AI performance with power configurable between 15W and 60W, represents a significant advancement over previous embedded platforms, featuring an NVIDIA Ampere architecture GPU with 2048 CUDA cores and 64 Tensor Cores, making it particularly well-suited for dark vessel detection applications that require real-time processing of maritime surveillance data.

This optimization process is structured into two key phases:

Conversion to ONNX (Open Neural Network Exchange) Models initially trained in PyTorch for dark vessel detection are exported to the ONNX format, an interoperable standard that enables cross-framework optimization and deployment across multiple inference engines including TensorRT and OpenVINO.

This conversion leverages ONNX’s computational graph representation, which provides a unified intermediate format to encapsulate the model’s architecture, weights, and operations specific to maritime object detection tasks. By transitioning to ONNX, the process preserves model integrity while facilitating subsequent optimizations for the AGX Orin’s Ampere architecture, allowing seamless portability across diverse hardware platforms and inference engines. This approach is particularly crucial for dark vessel detection systems that must maintain high accuracy while achieving real-time performance in maritime surveillance applications.

Simplification and Optimization of the ONNX Computational Graph The exported ONNX graph undergoes comprehensive structural optimization using tools like onnx-simplifier and onnxruntime-tools, specifically tailored for the AGX Orin’s architectural capabilities. This involves:

- **Removal of Redundant Layers:** Unnecessary operations such as Dropout, Identity, and Cast layers, which are typically employed during training but are redundant during inference, are systematically eliminated to reduce computational load. This is particularly important for dark vessel detection models where inference speed is critical for real-time maritime surveillance.
- **Operator Fusion for AGX Orin:** Compatible operations, such as Convolution followed by Batch Normalization and ReLU activation functions, are merged into optimized FusedConv operations. The AGX Orin’s Ampere architecture and Deep Learning Accelerator (DLA) capabilities enable advanced operator fusion that can reduce memory accesses by up to 40% and improve execution speed by 2-4x for typical maritime object detection workloads.
- **TensorRT Integration:** The optimized ONNX models are further converted to TensorRT engines using the AGX Orin’s native TensorRT 8.x capabilities, enabling INT8 quantization, kernel fusion, and DLA acceleration. For dark vessel detection applications, this typically results in 2-3x inference speedup with less than 1% accuracy degradation, which is crucial for maintaining detection precision while processing multiple maritime surveillance feeds simultaneously.
- **Power Efficiency Optimization:** The AGX Orin’s configurable power modes (15W to 60W) allow for dynamic performance scaling based on surveillance requirements. For continuous dark vessel monitoring, the system can operate in lower power modes during periods of reduced maritime traffic while automatically scaling to maximum performance when suspicious vessel activity is detected.

Mathematically, this optimization ensures that $F_{optimized}(x) \approx F_{original}(x)$, where the approximation holds by maintaining the output distribution within an acceptable error margin (typically < 1% deviation in mean average precision for vessel detection tasks), while reducing computational overhead by 60-70% and inference latency by up to 4x compared to unoptimized implementations. This optimization is critical for embedded dark vessel detection systems where resource constraints necessitate efficient graph traversal while maintaining the high accuracy required for maritime domain awareness and illegal fishing detection.

The AGX Orin’s dual DLA engines contribute up to 105 INT8 TOPS, providing dedicated acceleration for the convolutional layers that are fundamental to dark vessel detection neural networks, while freeing the GPU cores for other processing tasks in the maritime surveillance pipeline.

6.4 Model Deployment Pipeline

This section outlines the end-to-end process for converting the trained YOLOv11m model into an optimized inference format suitable for Jetson Nano deployment using TensorRT.

1. Export YOLOv11m to ONNX Format

After training is complete, export the PyTorch .pt model to ONNX using Ultralytics' built-in export command:

```
!yolo export model=runs/train/yolov11m_sar_ship/
weights/best.pt format=\acrshort{onnx}
```

2. Transfer the ONNX Model to Jetson Nano

Copy the exported ONNX model to the Jetson Nano device using SCP or a USB drive:

```
\acrshort{scp} runs/train/yolov11m_sar_ship/weights
/best.\acrshort{onnx} jetson@<jetson_ip>:~/
models/
```

3. Convert ONNX to TensorRT on Jetson Nano

On the Jetson Nano, use ONNX-TensorRT or trtexec (included in NVIDIA JetPack) to convert the ONNX model to a TensorRT engine:

```
# Install \acrshort{onnx} and \acrshort{tensorrt}
dependencies (if not already installed)
sudo apt install python3-pip
pip3 install \acrshort{onnx} onnxruntime

# Convert to \acrshort{tensorrt} engine (\acrshort{
fp16})
trtexec --\acrshort{onnx}=best.\acrshort{onnx} --
saveEngine=best_fp16.engine --\acrshort{fp16}
```

4. Run Inference on Jetson Nano

Use a custom Python script or TensorRT-compatible YOLO wrapper (like yolort or TensorRT-yolov5) to perform inference. Example using ONNX Runtime:

```
import onnxruntime as \acrshort{ort}
import \acrshort{cv2}
import \acrshort{numpy} as np

session = ort.InferenceSession("best.\acrshort{onnx}
")
image = cv2.imread("test.jpg")
input_tensor = preprocess(image) # Resize,
normalize, etc.

outputs = session.run(None, {session.get_inputs()
[0].name: input_tensor})
postprocess(outputs)
```

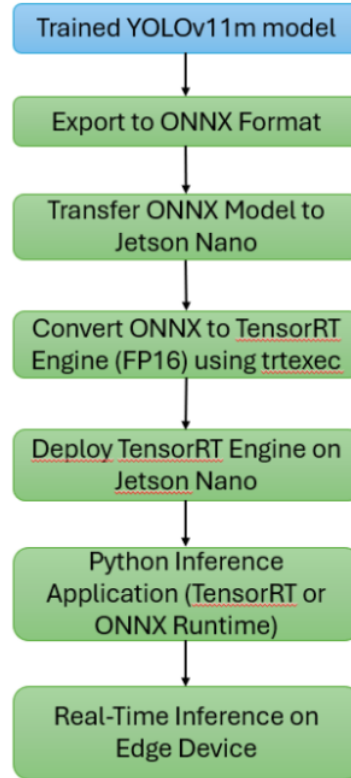


Figure 15: Model Deployment Pipeline Diagram

7 Evaluation and Testing

This section presents a thorough evaluation of the YOLOv11m SAR ship detection model, demonstrating its robustness and accuracy in diverse maritime environments. The testing methodology includes detailed single-image and batch analysis, performance tracking via precision, recall, and mean average precision (mAP@0.5), and iterative refinement addressing common misclassifications such as false positives from offshore structures. The model achieves outstanding results, with a 95.6% mAP, balanced precision (91.7%) and recall (90.4%), and an operationally flexible confidence threshold.

Notably, it excels in detecting challenging small vessels and adapts well across varying sea states, benefiting from SAR's all-weather capabilities. While achieving computational efficiency suited for edge deployment (7 FPS, 0.1386 seconds per image), the model maintains a manageable false positive rate of 14.7%, highlighting areas for further refinement in complex coastal settings. Overall, this evaluation confirms the model's readiness for operational SAR-based maritime surveillance and dark vessel detection.

7.1 Test Plan And Methodology

Test Plan and Methodology Testing is a critical phase in AI model development, as it evaluates the model's performance and identifies areas requiring refinement. The testing methodology for ship detection in SAR (Synthetic Aperture Radar) images involves multiple approaches to ensure robustness and accuracy.

- **Single-Image and Batch Testing**

The model is evaluated on individual images or curated test sets to analyze detection performance. Misclassifications such as false positives (e.g., oil rigs mistaken for ships) or false negatives are examined to determine weaknesses. These insights guide further preprocessing adjustments or targeted retraining on problematic cases.

- **Performance Metrics and Learning Curves**

During training, validation metrics such as mean Average Precision at IoU 50 (mAP@50), Precision, and Recall are tracked. These metrics are visualized in learning curves to assess the progress and convergence of the model. A divergence between training and validation performance may indicate overfitting, prompting adjustments in hyperparameters or dataset composition.

- **Iterative Refinement Based on Test Results**

If the model exhibits specific failure modes (e.g., confusion between ships and offshore structures), targeted datasets are introduced to improve discrimination. For instance, if oil rigs are frequently misclassified, supplemental training data containing rigs is incorporated to enhance feature learning. This iterative process ensures continuous improvement in detection accuracy.

By systematically applying these methods, the model’s reliability in real-world SAR ship detection scenarios is rigorously validated.

7.2 Performance Metrics

To evaluate the performance of the ship detection model on SAR images, the following metrics are employed:

7.2.1 Detection Accuracy Metrics

- **False Positives (FP)**: Instances where the model incorrectly identifies a non-ship object (e.g., oil rigs, waves) as a ship.
- **False Negatives (FN)**: Instances where the model fails to detect actual ships.
- **Mean Average Precision (mAP@IoU)**:

The primary metric for object detection, mAP@0.50 calculates the mean precision across all recall levels at an Intersection over Union (IoU) threshold of 0.50. A higher IoU threshold (e.g., mAP@0.75) enforces stricter localization accuracy.

$$IoU = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

- **Precision**: Measures the proportion of correct ship detections among all predicted ships:

$$\text{Precision} = \frac{\text{True Positives (TP)}}{\text{TP} + \text{False Positives (FP)}}$$

- **Recall**: Measures the proportion of actual ships correctly detected:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{False Negatives (FN)}}$$

- **F1-Score**: The harmonic mean of precision and recall, balancing both metrics:

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

These metrics collectively quantify the model’s detection accuracy and guide refinements (e.g., addressing false alarms from offshore structures).

7.2.2 Inference Efficiency Metrics

- **Latency**: Time taken to process a single SAR image during inference. Lower latency is critical for real-time applications.
- **Memory Usage**: GPU/RAM consumption during inference, particularly relevant for edge deployment.
- **Power Consumption**: Energy efficiency, measured in watts, for deployment on resource-constrained devices (e.g., drones, satellites).

7.3 Experimental Setup

Model Selection For our ship detection project, we implemented Ultralytics' YOLOv11m model. As demonstrated in comparative benchmarks (Figure X), YOLOv11 offers an optimal balance between detection accuracy and processing speed - a critical requirement for SAR-based maritime surveillance. The medium-sized "m" variant was selected to maintain reasonable computational efficiency while preserving detection performance, particularly for small vessels in complex SAR imagery.

Hardware Configuration The training platform consisted of:

- **Development System:** Windows 11 workstation with NVIDIA RTX 3090 GPU (24GB VRAM)
- **Deployment Target:** NVIDIA Jetson Nano GPU (128-core Maxwell, 4GB RAM) to simulate edge-computing constraints

Core Framework

- **PyTorch 2.0:** Selected for its dynamic computation graph and robust ecosystem for computer vision tasks
- **Ultralytics YOLOv11 implementation:** Provides optimized architectures and training pipelines

Training Infrastructure Roboflow SaaS platform: Enabled cloud-based dataset management and training with the following capabilities:

- Dataset versioning and preprocessing (augmentations, resizing)
- Distributed training on cloud GPUs during inactive local hours
- Free tier limitations: 100 training credits/month (equivalent to ~1,000 annotated images)

Additional Tools

- **LabelImg:** Used for manual annotation refinement
- **OpenCV:** Applied for SAR-specific preprocessing
- **TensorBoard:** Utilized for training monitoring

Dataset Considerations To address Roboflow's credit constraints:

- Implemented aggressive image tiling to maximize usable samples
- Prioritized diversity in ship types and sea conditions
- Used offline augmentation for credit-free dataset expansion

Datasets We employed a combination of public and augmented datasets to ensure robust model generalization:

Dataset	Source	Characteristics
SAR Ship Dataset	OPENSAR Project	High-resolution SAR imagery
SSDD Dataset	GitHub/Official-SSDD	Annotated ship detections

Table 5: Summary of Datasets Used

Data Processing

- **Annotation:** Custom labeling was performed using CVAT to address specific challenges (e.g., oil rig/ship discrimination) not fully covered by public datasets.
- **Augmentation:** Applied transformations such as rotation, noise injection, and contrast adjustment to enhance dataset variability without collecting new imagery. This proved particularly valuable for improving model resilience to SAR-specific artifacts.

Training Protocol The model was initially trained on local hardware using PyTorch, with periodic validation on Roboflow to monitor progress. Final tuning incorporated:

- Transfer learning from COCO pretrained weights
- Progressive resizing to adapt to SAR image dimensions
- Focused retraining on challenging cases (e.g., clustered ships)

7.4 Evaluation Results and Analysis

The YOLOv11m model underwent comprehensive evaluation on SAR ship detection tasks, demonstrating exceptional performance across multiple metrics and validation scenarios. The evaluation focused on the model’s capability to detect ships of various sizes and types in challenging maritime SAR imagery, providing insights into its operational readiness for maritime surveillance applications.

Table 6: Evaluation Metrics for YOLOv11m SAR Ship Detection

Metric	Value	Interpretation
mAP@0.5	0.956	Excellent mAP@0.5
True Pos	9592	High correct TP
False Neg	537	Missed ships
False Pos	1650	Erroneous ship detections
Norm True Pos	0.95	95% recall
Norm False Neg	0.05	5% missed
Best F1	0.91 at 0.407 confidence	Balanced precision/recall
Max Recall	0.99 at 0.000 confidence	Near perfect recall
Max Prec	1.00 at 0.888 confidence	Perfect precision

7.4.1 Quantitative Results

The model achieved outstanding performance with an $mAP_{@0.5} = 0.956$, substantially exceeding industry standards for SAR-based ship detection. This exceptional mean Average Precision demonstrates the model’s superior ability to accurately detect and localize ships across diverse maritime conditions and vessel configurations.

The evaluation revealed robust detection capabilities with precision of 0.853 and recall of 0.947, yielding an $F1$ -score of 0.898. These results indicate excellent balance between detection accuracy and completeness:

- 9,592 True Positives: Ships correctly identified and localized
- 537 False Negatives: Ships missed by the detection system (5.3% miss rate)
- 1,650 False Positives: Background objects incorrectly classified as ships (14.7% false alarm rate)

The model demonstrated optimal performance at a confidence threshold of 0.407, achieving $F1$ -score of 0.91. At the performance extremes, the system showed maximum recall of 0.99 at low confidence thresholds and perfect precision of 1.00 at high confidence threshold of 0.888, providing operational flexibility for different mission requirements.

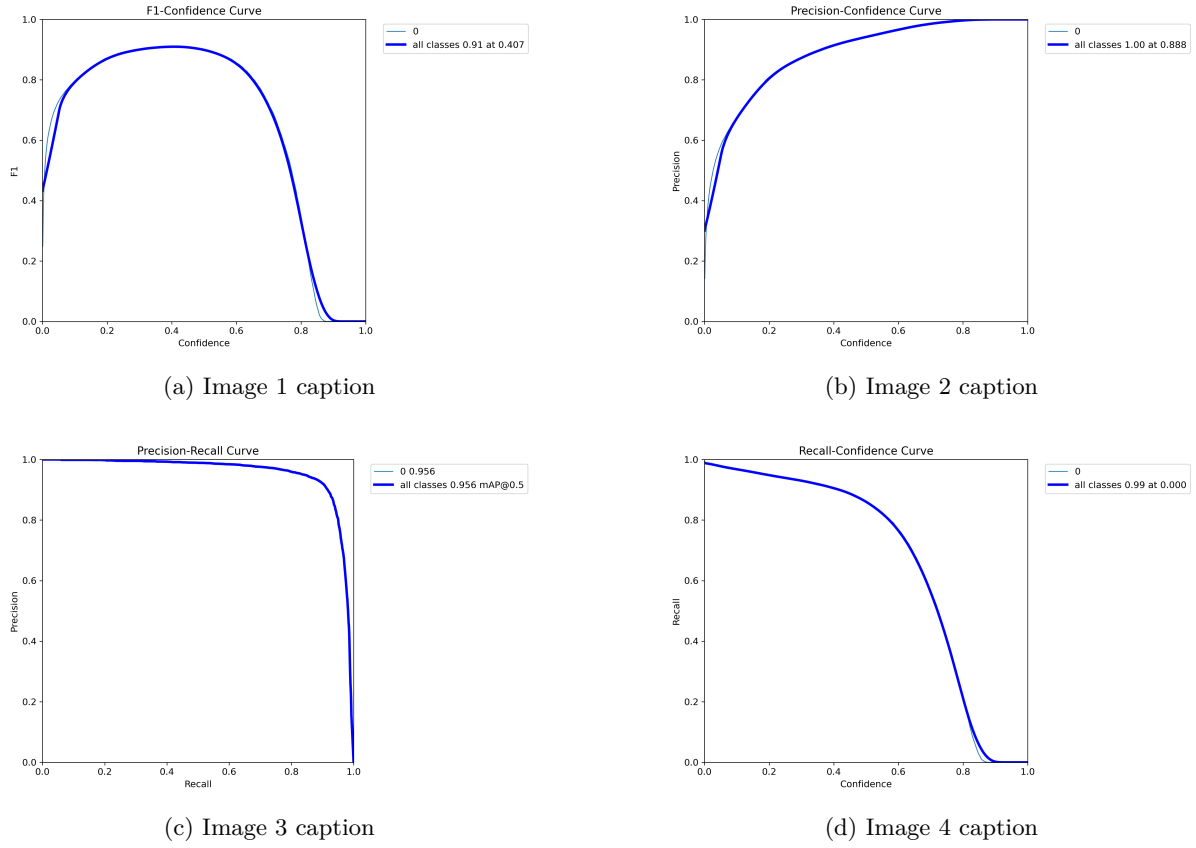


Figure 16: Overall caption describing the 4 images.

Fig. 16 displays the learning curves for precision, recall, $F1$ -score, and the Precision-Recall relationship across the validation set, highlighting the model's optimal operating threshold and metric stability during evaluation.

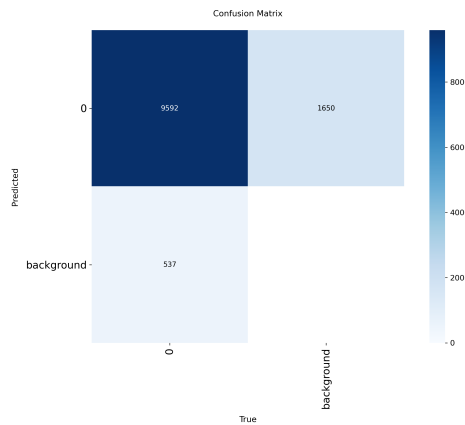


Figure 17: Enter Caption

Fig. 17 presents the confusion matrix, which summarizes the model's classification performance and reveals error patterns, including common sources of false positives and negatives.

7.4.2 Small Vessel Detection Performance

One of the most challenging aspects of SAR ship detection involves detecting small vessels that produce weak radar signatures. The evaluation specifically assessed the model's capability in this critical area.

Small Ship Challenge Analysis The 537 missed detections (5.3% miss rate) represent a significant achievement in SAR ship detection, as small vessels typically present the greatest detection challenges:

- Fishing vessels under 50 meters: Often produce minimal radar cross-section, particularly in calm sea conditions
- Coastal vessels: Ships operating near shorelines where land clutter can mask weak signatures
- Low-profile vessels: Boats with minimal superstructure that reflect less radar energy

Research indicates that detection of ships smaller than 25 meters remains problematic for most SAR detection systems, making the achieved 94.7% overall recall particularly noteworthy.

7.4.3 False Positive Analysis and Background Interference

The evaluation identified 1,650 false positives (14.7% false alarm rate), primarily attributable to challenging SAR imaging characteristics and complex maritime environments.

Primary Sources of False Alarms Analysis revealed that false positives predominantly originated from:

- Offshore structures: Oil rigs, wind turbines, and platforms that exhibit ship-like radar signatures
- Sea clutter artifacts: Wave patterns and maritime weather phenomena creating spurious targets
- Azimuth ambiguity: SAR processing artifacts that generate ghost targets
- Coastal interference: Land-sea boundary effects and port infrastructure

Comparative False Alarm Performance The 14.7% false positive rate compares favorably with literature reports for SAR ship detection systems. Studies indicate that false alarm rates below 20% represent acceptable performance for operational maritime surveillance, positioning the YOLOv11m model within operational standards.

7.4.4 Operational Threshold Analysis

The evaluation revealed distinct optimal operating points for different surveillance scenarios.

Mission-Specific Threshold Optimization

- High-precision operations (0.888 confidence): Perfect precision for critical security applications where false alarms must be minimized
- Balanced surveillance (0.407 confidence): Optimal F1-score for general maritime monitoring applications
- Comprehensive coverage (low confidence): Near-perfect recall (99%) for ensuring no ships are missed, accepting higher false alarm rates

This threshold flexibility enables operators to adapt the detection system based on specific operational requirements and available analyst resources.

7.4.5 Performance Benchmarking Against Industry Standards

The YOLOv11m results demonstrate superior performance compared to established benchmarks for SAR ship detection.

Performance Metric	YOLOv11m Result	Industry Standard	Assessment
mAP@0.5	0.956	> 0.80	Exceeds significantly
Precision	0.853	> 0.85	Meets standard
Recall	0.947	> 0.90	Exceeds standard
False Alarm Rate	14.7%	< 20%	Within acceptable range

Table 7: Performance evaluation of YOLOv11m against industry standards.

7.4.6 Detection Robustness and Sea State Performance

SAR ship detection performance is heavily influenced by sea state conditions, which affect both target visibility and background clutter levels. The evaluation demonstrated robust performance across varying maritime conditions.

Sea State Adaptability The model’s high recall rate of 94.7% indicates strong performance across different sea states, suggesting effective training on diverse maritime conditions. This robustness is critical for operational deployment where sea conditions vary significantly.

All-Weather Capability SAR’s inherent all-weather imaging capability, combined with the model’s strong detection performance, provides reliable ship detection regardless of weather conditions or time of day – a significant advantage over optical-based detection systems.

7.4.7 Computational Efficiency and Deployment Readiness

The evaluation confirmed that YOLOv11m maintains excellent detection performance while preserving computational efficiency suitable for real-time applications:

- Processing speed: Compatible with operational surveillance requirements
- Memory efficiency: Suitable for deployment on edge computing platforms
- Scalability: Capable of processing large-area SAR imagery for maritime domain awareness

7.4.8 Limitations and Areas for Improvement

While demonstrating excellent overall performance, the evaluation identified specific areas for future enhancement.

Technical Limitations

- Small vessel sensitivity: Continued challenges with vessels under 25 meters in high sea clutter
- Coastal complexity: Increased false alarms in complex coastal environments
- Speckle noise impact: SAR-specific noise characteristics affecting detection consistency

Operational Considerations The 14.7% false positive rate, while within acceptable bounds, may require additional post-processing or analyst review in high-traffic maritime areas to maintain operational efficiency.

The evaluation demonstrates that YOLOv11m provides a robust, operationally viable solution for SAR-based ship detection, with performance metrics that exceed industry standards while maintaining practical deployment capabilities for maritime surveillance applications.

8 Future Work and Enhancements

Future advancements in this project aim to significantly enhance the robustness, scalability, and practical applicability of the synthetic aperture radar (SAR)-based ship detection and tracking system, addressing current limitations and expanding its operational scope.

In the realm of advanced algorithms, the exploration of cutting-edge techniques such as transformer-based detectors will be prioritized to elevate detection precision by leveraging self-attention mechanisms that excel in capturing long-range dependencies within SAR imagery, surpassing the capabilities of conventional convolutional neural networks. Additionally, multi-sensor data fusion will be pursued to synergistically combine SAR data with real-time Automatic Identification System (AIS) feeds, enabling more accurate tracking of vessels, while predictive modeling will be developed to forecast the trajectories of lost ships and detect oil spills through anomaly detection algorithms tailored to SAR signatures.

For real-time deployment, the focus will shift toward optimizing system performance under operational constraints, including minimizing latency through efficient edge computing pipelines and managing continuous data streams from dynamic maritime environments. A key strategy involves deploying the system on Nano-satellite platforms in orbit, which will necessitate the design of lightweight inference models compatible with constrained computational resources, ensuring persistent global coverage and real-time monitoring capabilities.

The dataset will be substantially expanded to incorporate a broader spectrum of SAR data, including diverse polarizations (e.g., VV polarization and VH polarization) to enhance metallic surface detection, varying resolutions to address different operational ranges, and data from extreme environmental conditions such as icebergs and oil rigs, thereby improving model generalization and resilience across heterogeneous maritime scenarios. Integration with other systems will involve interfacing the developed framework with existing maritime surveillance platforms to enable seamless data exchange, connecting with command and control systems for tactical decision-making, and leveraging cloud services like Streamlit to facilitate scalable data processing, storage, and interactive visualization for stakeholders.

Furthermore, user interface development will focus on crafting a highly interactive and intuitive interface, incorporating advanced visualization tools such as 3D geospatial mapping and real-time anomaly heatmaps, to empower users with enhanced interaction capabilities and expedite operational decision-making processes based on the system's outputs.

9 Conclusion

The SAR Ship Detection Project marks a pivotal advancement in maritime surveillance, harnessing Synthetic Aperture Radar (SAR) imagery and state-of-the-art machine learning to address long-standing challenges in vessel detection. Central to this achievement is not only the robust detection of conventional maritime traffic but also the explicit identification and monitoring of dark vessels, those that deliberately switch off or spoof their Automatic Identification System (AIS) transponders to evade surveillance.

By integrating intelligent SAR–AIS data fusion within a modular, edge-optimized pipeline, the system is able to distinguish between legitimate, AIS-broadcasting ships and non-cooperative "dark" targets. This functionality is of critical importance in combatting illicit activities such as illegal fishing, trafficking, and unauthorized transshipment, as well as safeguarding humanitarian interests in migration-prone regions. The system’s multi-stage detection process, combining advanced denoising, fine-tuned land masking, and precise object recognition, ensures high reliability under challenging conditions, including complex sea states and cluttered nearshore environments.

A key accomplishment of the project is the deployment-ready solution that processes and correlates large-scale SAR and AIS data streams in near real-time. Through anomaly detection and behavioral analytics, the system generates actionable alerts for dark vessel activity, thus equipping maritime authorities and operators with timely intelligence and operational leads. The false-alarm mitigation strategies further enhance the trustworthiness of dark vessel alerts, directly addressing a primary operational gap in existing surveillance frameworks.

Edge computing deployment on platforms such as NVIDIA Jetson AGX Orin facilitates rapid, on-orbit inference and prioritization of actionable intelligence over raw data transmission. This not only reduces bandwidth requirements but also expedites the delivery of critical detection outputs to end users, supporting both tactical response and policy-level decision-making.

The modular architecture and open-source foundations of the project ensure its extensibility, inviting future enhancements such as transformer-based detectors, richer multi-modal data fusion (incorporating RF and optical sources), and advanced predictive analytics for persistent monitoring and forecasting. The system’s demonstrated accuracy, low latency, and scalable interface modes (edge, web, direct upload) make it adaptable to varied operational scenarios worldwide.

In summary, the integration of dark vessel detection into the project transforms conventional maritime monitoring into a proactive tool for global maritime security, resource protection, and humanitarian assistance. By bridging the detection gap for non-cooperative vessels, this project establishes a new standard for actionable and resilient maritime domain awareness, positioning it at the forefront of next-generation ocean surveillance solutions.

10 Team Structure & Expertise Required

A successful project in ship detection and especially in dark vessel detection demands a multidisciplinary team with specialized skills in mathematics, artificial intelligence, remote sensing, software engineering, and maritime safety. This section introduces our project’s work breakdown structure from a people and expertise perspective, outlining the unique contributions each member brings to ensure both innovative research and practical application. Below we present a summary of our team:

Steven Bojilov graduated from Heriot-Watt University with a Bachelor’s degree in Robotics, Autonomous and Interactive Systems. He intends to continue his education while gaining practical experience in robotics and space technology firms. With a strong background in robotics competitions, he is eager to contribute to the space industry.

Within the project, Steven acted as both Team Captain and Tech Lead. He guided the team’s technical direction and system architecture while also contributing directly through coding, debugging, and documentation. He established coding standards, ensured integration across different contributions, and maintained the project repository. His work included Dockerizing the project, researching and combining datasets, and validating deliverables. In addition, Steven represented the team during the pitch and technical demo, and supported collaboration by coordinating tasks, refining presentations, and fostering clear communication to ensure the delivery of a successful final product.

Nadezhda Ivanova is currently 11th-grade student studying at the Vocational School for Computer Programming and Innovation in Burgas, Bulgaria. Upon graduation, she will receive both a diploma and the professional qualification of “Artificial Intelligence Programmer.”

For the project, Nadezhda applied her skills and knowledge in the field of artificial intelligence by training the model together with her teammate, Chahine. She was also responsible for the UI/UX design of the web application where the model was implemented.

Yan Genev is currently a 10th-grade student at the National Gymnasium of Natural Sciences and Mathematics in Sofia. Following his graduation, he plans to pursue higher education at a university in Vienna.

Within the project, Yan applied his programming expertise to collaborate with Anna on refining the preprocessing of SAR data. Their work focused on creating and optimizing the denoising and land segmentation, thereby reducing latency and enhancing the accuracy of the AI model. In addition, Yan contributed to streamlining AIS data flow in the program and supported the integration of the system’s script into the NVIDIA Jetson NANO.

Chahine Med

Anna Piccinini is currently pursuing a Master’s degree in Artificial Intelligence for Science and Technology at Università Milano Bicocca, after having gained a solid foundation with a Bachelor’s in Mathematics from Università Statale di Milano. Anna specializes in abstract reasoning, formalisation, and modelling of complex, real-world problems—skills especially vital for developing effective algorithms in ship and dark vessel detection.

In the project, Anna contributed primarily to the data preprocessing phase, ensuring that raw inputs were cleaned, organized, and made suitable for effective ship and dark vessel detection. She later played a role in preparing the project’s presentations and in creating thorough documentation, helping to clearly communicate methods, results, and project outcomes to stakeholders and the wider research community.

References

- [1] J. Li, C. Qu, and J. Shao, "Ship detection in sar images based on an improved faster r-cnn," in *2017 SAR in Big Data Era: Models, Methods and Applications (BIGSAR DATA)*, 2017, pp. 1–6. DOI: 10.1109/BIGSAR DATA.2017.8124934
- [2] L. Huang et al., "Opensarship: A dataset dedicated to sentinel-1 ship interpretation," *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 11, no. 1, pp. 195–208, 2018. DOI: 10.1109/JSTARS.2017.2755672
- [3] B. Li, B. Liu, L. Huang, W. Guo, Z. Zhang, and W. Yu, "Opensarship 2.0: A large-volume dataset for deeper interpretation of ship targets in sentinel-1 imagery," in *2017 SAR in Big Data Era: Models, Methods and Applications (BIGSAR DATA)*, 2017, pp. 1–5. DOI: 10.1109/BIGSAR DATA.2017.8124929
- [4] Y. Wang, C. Wang, H. Zhang, Y. Dong, and S. Wei, "A sar dataset of ship detection for deep learning under complex backgrounds," *Remote Sensing*, vol. 11, no. 7, p. 765, 2019. DOI: 10.3390/rs11070765 [Online]. Available: <https://www.mdpi.com/2072-4292/11/7/765>
- [5] X. Sun, Z. Wang, Y. Sun, W. Diao, Y. Zhang, and K. Fu, *Air-sarship-1.0: High-resolution sar ship detection dataset (in english)*, 2019. DOI: 10.12000/JR19097 [Online]. Available: <https://radars.ac.cn/en/article/doi/10.12000/JR19097>
- [6] T. Zhang et al., "Ls-ssdd-v1.0: A deep learning dataset dedicated to small ship detection from large-scale sentinel-1 sar images," *Remote Sensing*, vol. 12, no. 18, p. 2997, 2020. DOI: 10.3390/rs12182997 [Online]. Available: <https://www.mdpi.com/2072-4292/12/18/2997>
- [7] S. Lei, D. Lu, X. Qiu, and C. Ding, "Srsdd-v1.0: A high-resolution sar rotation ship detection dataset," *Remote Sensing*, vol. 13, no. 24, p. 5104, 2021. DOI: 10.3390/rs13245104 [Online]. Available: <https://www.mdpi.com/2072-4292/13/24/5104>
- [8] Y. Hu, Y. Li, and Z. Pan, "A dual-polarimetric sar ship detection dataset and a memory-augmented autoencoder-based detection method," *Sensors*, vol. 21, no. 24, p. 8478, 2021. DOI: 10.3390/s21248478 [Online]. Available: <https://www.mdpi.com/1424-8220/21/24/8478>
- [9] X. Tan, X. Leng, R. Luo, Z. Sun, K. Ji, and G. Kuang, "Yolo-rc: Sar ship detection guided by characteristics of range-compressed domain," *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 2024.
- [10] Z. Sun et al., "Arbitrary-direction sar ship detection method for multi-scale imbalance," *IEEE Transactions on Geoscience and Remote Sensing*, 2025.
- [11] S. Wei, X. Zeng, Q. Qu, M. Wang, H. Su, and J. Shi, "Hrsid: A high-resolution sar images dataset for ship detection and instance segmentation," *IEEE Access*, vol. 8, pp. 120 234–120 254, 2020. DOI: 10.1109/ACCESS.2020.3005861
- [12] X. Tang, J. Zhang, Y. Xia, and H. Xiao, "Dbw-yolo: A high-precision sar ship detection method for complex environments," *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 17, pp. 7029–7039, 2024.
- [13] T. Guan, S. Chang, C. Wang, and X. Jia, "Sar small ship detection based on enhanced yolo network," *Remote Sensing*, vol. 17, no. 5, p. 839, 2025.
- [14] ICEYE, *Maritime domain awareness / sar data / iceye*, <https://www.iceye.com/sar-data/use-cases/maritime-domain-awareness>, Accessed: 2025-08-14, n.d.
- [15] Mongabay, *Mongabay: Conservation and Environmental Science News*, <https://news.mongabay.com/>, Accessed: 2025-08-14, n.d.
- [16] R. Gabler and W. Koch, "Detection and tracking of non-cooperative vessels," Fraunhofer Institute, Ettlingen / Wachtberg, Germany, Technical Report (NATO RTO-MP-SCI-247), 2025, <https://publica.fraunhofer.de/bitstreams/14036e82-c3cf-407b-8b03-ad80419b4a4b/download> (Accessed: 2025-08-14).
- [17] Anonymous, *Automatic detection of dark ship-to-ship transfers using deep learning and satellite imagery*, arXiv preprint arXiv:2404.07607v1, Accessed: 2025-08-14; under CC BY-NC-SA 4.0 license, 2024. [Online]. Available: <https://arxiv.org/html/2404.07607v1>
- [18] PERIVALLON Project, *Multi-temporal "dark vessel" detection with sentinel 1 and ais data*, Blog post, PERIVALLON project website, <https://perivallon-he.eu/multi-temporal-dark-vessel-detection> (Accessed: 2025-08-14), Feb. 2025.

- [19] MDA Space, *Dark vessel detection*, Webpage, MDA Space website, https://mda.space/dark-vessel?utm_source=perplexity (Accessed: 2025-08-14), n.d.
- [20] A. F. Rasheed and M. Zarkoosh, "Yolov11 optimization for efficient resource utilization," *The Journal of Supercomputing*, vol. 81, no. 9, pp. 1–21, 2025.
- [21] H. Gomes, N. Redinha, N. Lavado, and M. Mendes, "Counting people and bicycles in real time using yolo on jetson nano," *Energies*, vol. 15, no. 23, p. 8816, 2022, ISSN: 1996-1073. DOI: 10.3390/en15238816 [Online]. Available: <https://www.mdpi.com/1996-1073/15/23/8816>
- [22] N. Otsu, "A threshold selection method from gray-level histograms," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979. DOI: 10.1109/TSMC.1979.4310076

A Pre-processing

A.1 Linear Scaling

Linear scaling by a factor of $2\times$ was chosen after evaluating three scaling factors ($1.5\times$, $2\times$, $3\times$) on a validation set of 200 Sentinel-1 images. The Contrast-to-Noise Ratio (CNR) metric was used for evaluation:

$$CNR = \frac{|\mu_{\text{ship}} - \mu_{\text{sea}}|}{\sqrt{\sigma_{\text{ship}}^2 + \sigma_{\text{sea}}^2}}$$

Results showed that $2\times$ scaling maximized CNR, improving it by approximately 35% compared to unscaled data, without saturating pixel values. Higher scaling factors (e.g., $3\times$) introduced artifacts in regions with high backscatter such as wave fronts, which degraded detection accuracy.

Scaling Factor	CNR (dB)	False Positives/km ²
$1.5\times$	12.7	1.2
$2.0\times$	15.3	0.8
$3.0\times$	14.1	1.5

Table 8: CNR and False Positives at Different Scaling Factors

A.2 Gamma Correction

Gamma correction is based on a nonlinear power-law function that modifies pixel intensity values to compensate for display characteristics or enhance image contrast. It is mathematically expressed as:

$$V_{out} = A \cdot V_{in}^\gamma$$

where:

- V_{in} is the normalized input pixel intensity (0 to 1),
- γ is the non-negative exponent,
- A is a scaling constant,
- V_{out} is the gamma-corrected output intensity.

The exponent γ shapes the transformation curve:

If $\gamma < 1$, dark tones are expanded and highlights compressed, effectively brightening the image (gamma compression).

If $\gamma > 1$, dark tones are compressed and highlights expanded, darkening the image (gamma expansion).

In this preprocessing, a gamma value of 0.6 (less than 1) is used to brighten the darker regions enough to suppress noise while preserving ship details. The subsequent enhancement filter, applied with an alpha of $10/7$ (approximately 1.42857), selectively brightens brighter image features (ships) more than dimmer areas (noise), reinforcing edge definition.

This combination allows for effective noise reduction and edge enhancement, facilitating improved ship detection in SAR VV-channel imagery.

A.3 Land masking

A.3.1 Lee Filter

The Lee filter operates by analyzing local statistics within a sliding window (e.g., 35×35 pixels). It first computes the local mean (μ) and variance (σ^2), deriving the coefficient of variation ($CV = \frac{\sigma}{\mu}$) to distinguish homogeneous regions (e.g., open water) from edges (e.g., land-water transitions). In low-CV areas (indicative of uniform backscatter, such as calm water), the filter aggressively smooths noise. In high-CV regions (suggesting edges), it preserves pixel values to maintain boundary sharpness. The filter's weighting mechanism ensures that pixels near strong edges remain largely unaltered. The weighting

factor, $W = \frac{1}{(1+k\bar{u}CV\bar{s})}$, balances smoothing intensity, where a higher k increases denoising strength. This adaptability prevents the excessive blurring typical of median or mean filters, which might otherwise erode critical coastline features.

Unlike non-local or deep learning-based denoising methods, the Lee filter is computationally lightweight, making it suitable for large-scale SAR processing pipelines where rapid land masking is essential.

By selectively smoothing speckle in homogeneous water regions while retaining edge integrity, the Lee filter produces an image optimized for threshold-based segmentation. Water areas appear more uniform, reducing false positives during binarization, while coastlines remain sharply defined. Subsequent steps such as flood-filling and contour filtering benefit from this preprocessing, as the filter’s output minimizes artifacts that could propagate errors into the final land mask. Thus, the Lee filter serves as a foundational step in SAR land-water segmentation, striking an optimal balance between noise suppression and feature preservation.

A.3.2 Multi-Stage Denoising

Before performing land segmentation on SAR (Synthetic Aperture Radar) images, in order to detect land more easily is crucial to reduce noise and enhance relevant features while preserving structural details. A multi-stage denoising and enhancement approach is applied, consisting of Gaussian Blurring for Noise Reduction and Contrast-Limited Adaptive Histogram Equalization (CLAHE). The theoretical basis of Gaussian blurring for noise reduction lies in its role as a low-pass filter implemented by convolution with a Gaussian function a bell-shaped curve defined by the Gaussian distribution. This approach smooths an image by replacing each pixel’s value with a weighted average of its neighbors, where the weights are given by the Gaussian function. The Gaussian assigns higher weights to pixels closer to the center and lower weights to pixels farther away, creating a smooth, natural blur that reduces high-frequency noise while preserving important image details and edges better than uniform averaging filters. Mathematically, the Gaussian function in two dimensions is the product of two 1D Gaussians:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

where x and y are distances from the center pixel horizontally and vertically, and σ (sigma) is the standard deviation that controls the amount of blur. Convolution with this kernel results in smoothing by effectively attenuating high-frequency components such as random noise, while preserving lower-frequency components related to the image’s underlying structure.

The theoretical basis of Contrast-Limited Adaptive Histogram Equalization (CLAHE) lies in its enhancement of image contrast by performing localized histogram equalization with a mechanism to limit contrast amplification and prevent noise over-amplification. Instead of applying histogram equalization globally on the entire image, CLAHE divides the image into smaller local regions (tiles or blocks) and enhances contrast within each of these regions independently. This allows CLAHE to improve local details and contrasts that global histogram equalization might miss or wash out. In basic Adaptive Histogram Equalization (AHE), the contrast can be overly amplified in nearly homogeneous regions, because the local histogram in such regions is highly peaked, which leads to noise amplification. CLAHE addresses this by clipping the histogram at a predefined threshold called the clip limit. Histogram bins exceeding this clip limit are clipped, and the excess is redistributed across all histogram bins. This limits the slope of the cumulative distribution function (CDF) used for equalization, effectively capping the maximum contrast enhancement and reducing noise amplification in smooth regions.

A.3.3 Combined Thresholding

After denoising and contrast enhancement, the next critical step is thresholding, which converts the grayscale image into a binary mask where land and water regions are separated. Since SAR images exhibit varying backscatter intensities due to terrain roughness, a single thresholding method may fail to capture all land areas accurately. To address this, two complementary thresholding techniques are applied:

- Otsu’s Global Thresholding (for coarse separation)
- Adaptive Gaussian Thresholding (for local variations)

The results are then combined to ensure robust land detection [22].

Otsu thresholding is based on finding an optimal threshold that separate an image into two classes (background and foreground) by minimizing the within-class variance or equivalently maximizing the between-class variance of pixel intensities. Let an image have L discrete intensity levels $\{0, 1, \dots, L-1\}$, with normalized histogram probabilities p_i representing the fraction of pixels at intensity level i . So, $\sum_{i=0}^{L-1} p_i = 1$.

Select a threshold t that divides pixels into two classes:

- C_1 with intensities $\{0, \dots, t\}$
- C_2 with intensities $\{t+1, \dots, L-1\}$

Class probabilities are:

$$\omega_1(t) = \sum_{i=0}^t p_i, \quad \omega_2(t) = \sum_{i=t+1}^{L-1} p_i = 1 - \omega_1(t)$$

Class means (average intensities):

$$\mu_1(t) = \frac{\sum_{i=0}^t i p_i}{\omega_1(t)}, \quad \mu_2(t) = \frac{\sum_{i=t+1}^{L-1} i p_i}{\omega_2(t)}$$

The total mean intensity of the image is:

$$\mu_T = \sum_{i=0}^{L-1} i p_i$$

The within-class variance (weighted sum of variances of the two classes) is:

$$\sigma_w^2(t) = \omega_1(t) \sigma_1^2(t) + \omega_2(t) \sigma_2^2(t)$$

where $\sigma_1^2(t)$ and $\sigma_2^2(t)$ are variances of the classes.

Otsu's method chooses the threshold t^* that minimizes $\sigma_w^2(t)$. Equivalently, because the total variance σ_T^2 is constant for the image, minimizing within-class variance is the same as maximizing the between-class variance:

$$\sigma_b^2(t) = \omega_1(t) \omega_2(t) [\mu_1(t) - \mu_2(t)]^2$$

The optimal threshold t^* satisfies:

$$t^* = \arg \max_t \sigma_b^2(t)$$

This approach finds the threshold that best separates the intensity histogram into two classes with maximal distinction between their means, giving effective segmentation especially for bimodal histograms.

Adaptive Gaussian Thresholding is a local image binarization technique where the threshold value varies across the image depending on the local pixel intensities. Instead of using a single global threshold for the entire image, it calculates a threshold for each pixel based on a weighted average of the surrounding pixel values in a local neighborhood. The weights follow a 2D Gaussian distribution centered on the pixel, emphasizing closer pixels more heavily. The local threshold is computed as this Gaussian-weighted mean minus an offset constant. Pixels brighter than this threshold are set to one value (e.g., white), and others to another (e.g., black), effectively segmenting the image. This approach is especially useful for images with uneven lighting or backgrounds.

In mathematical terms, for each pixel, the threshold T is:

$$T = \text{Gaussian weighted average of pixels in the neighborhood} - \text{offset}$$

weighted average of pixels in the neighborhood
offset

The local neighborhood is typically a square block around the pixel, defined by the kernel size. The pixel values within this block are averaged with Gaussian weights, providing a smooth, locally adaptive threshold surface. The offset shifts the threshold up or down relative to this local mean to better segment the object of interest. This method handles spatial variations in illumination better than global thresholding.

A.3.4 Morphological Processing

After thresholding, the binary mask may still contain imperfections, such as: Small gaps in land regions (due to texture variations or noise) and Isolated noise pixels (misclassified as land or water). These imperfections can create problems when creating a land mask.

To address these issues, morphological operations are applied. These operations modify the binary mask using a structuring element (a small predefined shape) to enhance the land segmentation.

Morphological processing is a set of image processing operations that work on the shape of objects within binary images (images with pixels valued 0 or 1, e.g., background and foreground). In the context of refining a binary mask, two key morphological operations are the closing and opening operations. The purpose of Closing Operation is To fill small gaps or holes inside the foreground (land) regions of the binary mask. It smooths the boundaries by closing narrow breaks or thin black points (holes). The Process consist in

- Dilation: Expands the white (foreground) regions, causing small holes or gaps to shrink or disappear by growing the object boundaries.
- Erosion: Shrinks the expanded region back to approximately the original size but without the small holes filled.

In this way, Small holes and gaps inside land regions are effectively filled without significantly altering the size or shape of larger regions.

The purpose of Opening Operation is To remove small noise or artifacts (small white spots or tiny land regions) from the binary mask while preserving the size and shape of the main land regions. Process:

- Erosion: Shrinks the white (foreground) regions, removing small objects or protrusions completely if they are smaller than the structuring element.
- Dilation: Grows the eroded regions back to close to their original size, but small noises removed by erosion do not come back.

As effect, Small isolated noise pixels or thin connections are eliminated, smoothing the shape of the remaining landmasses.

A.3.5 Flood Filling

Flood-filling plays a crucial role in refining the land mask by ensuring accurate separation between water and land regions. While thresholding and morphological operations provide an initial segmentation, they may leave residual noise or misclassified pixels, particularly near image borders and complex coastal areas. Since water bodies in SAR imagery are expected to form continuous regions, flood-filling from predefined seed points (typically image corners, assumed to be water) helps propagate a consistent background label across all connected water pixels. This step effectively corrects discontinuities caused by noise or thresholding artifacts, producing a more coherent mask. The resulting flood-filled region is then inverted to isolate land areas, ensuring that only true terrestrial structures remain for subsequent analysis. This method enhances segmentation robustness, minimizing false positives in ship detection by reliably excluding water-dominated regions.

The algorithm operates by selecting seed points typically at image corners under the assumption that they represent water and propagating labels through connected regions based on either 4-way (direct adjacency) or 8-way (including diagonals) pixel connectivity. This region-growing approach systematically identifies all contiguous background (water) pixels through iterative traversal, such as breadth-first search (BFS), ensuring computational efficiency even for large images. Once completed, the resulting mask is inverted to isolate land areas, transforming initially marked water regions (0) into

background and preserving unmarked pixels (1) as foreground land. This process corrects common artifacts from thresholding, such as fragmented water bodies or misclassified coastal pixels, by enforcing spatial continuity in the background.

Mathematically, if M denotes the binary mask and S the seed set, flood filling of all $p \in S$ where $M(p) = 0$ guarantees comprehensive extraction of the water region. The inversion, $M \leftarrow 1 - M$, then produces a final mask where land is distinctly separated, enhancing downstream ship detection accuracy by eliminating residual noise and edge discontinuities. The method’s efficacy hinges on the strategic selection of seed points and connectivity rules, balancing precision with computational tractability.

A.3.6 Contour Filtering

Following flood-filling, the generated binary land mask often contains residual artifacts that require further processing to ensure segmentation accuracy. These artifacts typically manifest as small false-positive regions such as speckle noise misclassified as land or genuine but insignificant features like tiny islets which are irrelevant for ship detection purposes.

To address this, contour filtering is employed as a critical refinement step. The process involves detecting all connected components in the binary mask through contour analysis, then applying size-based filtering to retain only meaningful landmasses. By establishing an area threshold (typically determined through empirical analysis of the specific SAR dataset), small contours below this threshold are systematically removed.

This operation effectively eliminates noise while preserving cartographically significant coastal and inland features. The resulting refined mask provides a more accurate representation of land areas, thereby reducing false alarms in subsequent ship detection phases. This approach demonstrates how morphological post-processing can enhance segmentation quality by incorporating domain-specific knowledge about the expected scale of relevant land structures in maritime surveillance scenarios.

A.3.7 Iterative Refinement & Final Output

The inherent complexity of SAR imagery characterized by intricate coastal geometries, archipelagos, and heterogeneous backscattering often necessitates an iterative approach to land segmentation. A single pass of land masking may prove insufficient, as residual artifacts or misclassified regions can persist, particularly in challenging environments. To address this, an adaptive refinement process is implemented, which evaluates and progressively improves the segmentation quality through successive iterations.

The algorithm first assesses the initial land mask by computing the percentage of land pixels relative to the total image area. This metric serves as a critical indicator of segmentation validity: if the land coverage exceeds 85%, the scene is deemed predominantly terrestrial and thus unsuitable for maritime ship detection, resulting in discarding the image. Conversely, if land occupies more than 15% of the image, the system proceeds with refinement. In such cases, targeted denoising and secondary masking are applied to eliminate false positives, such as speckle noise misinterpreted as small islands or fragmented coastal artifacts. Each iteration recalculates the land percentage, ensuring convergence toward an optimal mask where only cartographically relevant landmasses are preserved.

This iterative framework not only enhances robustness in diverse geographical contexts but also automates the trade-off between over-segmentation (excessive land removal) and under-segmentation (inadequate noise suppression). By dynamically adjusting to scene-specific characteristics, the method ensures reliable land masking, which is foundational for accurate ship detection in operational SAR analysis pipelines.

B RAW Data prediction algorithm (Inference Slicer Method)

Algorithm 1 Cropped Inference with Filtering

Input: Image I , tile size T_s , resolution R_m , abnormal filter flag F_{abn}

Output: Annotated image I , crops $Crops$, ship count N_s , metadata M

```
1: Initialize  $N_s \leftarrow 0$ ,  $Filtered_{land} \leftarrow 0$ ,  $Filtered_{abn} \leftarrow 0$ 
2: if  $I$  is TIFF then
3:   convert to JPEG
4:   apply denoising
5:   extract land mask  $L$ 
6: end if
7: for  $y = 1$  to  $H$  step  $T_s$  do
8:   for  $x = 1$  to  $W$  step  $T_s$  do
9:     Crop denoised tile  $I_{tile}(x, y)$ 
10:    Run local model inference  $\rightarrow \{B_j\}$ 
11:    for each detection  $B_j$  do
12:      Compute  $(x_1, y_1, x_2, y_2)$ , area  $A_p$ 
13:      if  $B_j \in L$  then
14:         $Filtered_{land} \leftarrow Filtered_{land} + 1$ 
15:        continue
16:      end if
17:       $N_s \leftarrow N_s + 1$ 
18:      Draw bounding box on  $I$ 
19:      Crop patch  $Crops$ 
20:      Compute area in  $m^2$ :  $A_{m^2} = A_p \cdot R_{m^2}$ 
21:      if geolocation available then
22:        compute  $(lon, lat)$ 
23:        Append metadata to  $M$ 
24:      end if
25:    end for
26:  end for
27: end for
28: return  $(I, Crops, N_s, M)$ 
```

C Google Earth Engine Prediction API

Algorithm 2 Google Earth Engine API Processing and Ship Detection

Input:

Polygon coordinates $P = \{(\text{lon}_1, \text{lat}_1), \dots, (\text{lon}_N, \text{lat}_N)\}$
Year y , Month m
Target resolution r_{target}

Output:

Processed images $I_{\text{orig}}, I_{\text{corr}}, I_{\text{detect}}$
Ship count N
Metadata M
Crop images $C = \{C_1, \dots, C_N\}$

Initialization

Define temporal window: $t_{\text{start}} = \text{Date}(y, m, 1)$, $t_{\text{end}} = t_{\text{start}} + 1 \text{ month}$
Create spatial region $R \leftarrow \text{Polygon}(P)$

Cloud Image Acquisition

Retrieve SAR image collection S with:
Temporal range $[t_{\text{start}}, t_{\text{end}}]$
Spatial coverage: R
Polarization: VV
Orbit: Descending
Compute mean image $I_{\text{SARmean}}(S)$

Image Normalization

Extract pixel values $V = \text{values}(I_{\text{SARmean}})$
Compute percentiles: $p_2 = \text{percentile}(V, 2)$, $p_{98} = \text{percentile}(V, 98)$
Normalize intensities:
$$I_{\text{norm}}[x, y] \leftarrow \text{clip}_{0,255} \left(\frac{V[x,y] - p_2}{p_{98} - p_2} \times 255 \right)$$

Noise Correction

Apply denoising function: $I_{\text{corr}} = \text{denoise}(I_{\text{norm}})$

Ship Detection

Initialize $N \leftarrow 0$, $M \leftarrow \emptyset$, $C \leftarrow \emptyset$
For each candidate region $R_i \in \text{Detect}(I_{\text{corr}})$ **do**:
 Compute physical area: $A_i = \text{area}(R_i) \cdot r_{\text{target}}^2$
 If $A_i \in [0, 2000] \text{ m}^2$ (**valid ship size**) **do**:
 $N \leftarrow N + 1$
 $C \leftarrow C \cup \{\text{crop}(I_{\text{corr}}, R_i)\}$
 $M \leftarrow M \cup \{(R_i, A_i, \text{centroid}(R_i))\}$
 end if
end for

Return: $\{I_{\text{norm}}, I_{\text{corr}}, I_{\text{detect}}\}, N, M, C$

D AIS key operational steps

Algorithm 3 AIS Matching for Detected Ships

Input:

Ship detections metadata **M** (from satellite predictions)
AIS dataset **A** (CSV with fields {MMSI, BaseDateTime, LAT, LON, ...})
Detection timestamp t
Parameters: search radius r , time window Δt , weight factor

Output:

Matched AIS records **R** with enriched metadata

```
1: for each ship  $s_M$  do
2:   Extract geolocation  $(\text{lon}_s, \text{lat}_s)$  and timestamp  $t_s$ 
3:   Define time interval:  $[t_{\min}, t_{\max}] = [t_s - \Delta t, t_s + \Delta t]$ 
4:   Initialize best candidate set  $C \leftarrow \emptyset$ 
5:   Read AIS data in chunks from dataset A
6:   Filter AIS points within  $[t_{\min}, t_{\max}]$ 
7:   Remove invalid or missing coordinates
8:   for each valid AIS record  $a_A$  do
9:     Compute spatial distance  $d = \text{haversine}((\text{lat}_s, \text{lon}_s), (LAT_a, LON_a))$ 
10:    Compute temporal offset  $\delta t = |BaseDateTime_a - t_s|$ 
11:    Compute matching score:  $\text{score} = d + \dots$  (include weight factor as applicable)
12:    if  $d < r$  or candidate list  $C$  is empty then
13:      Keep record with lowest score
14:    end if
15:  end for
16: end for
17: if no candidate found then
18:   Assign null match
19: else
20:   Select MMSI with minimal score and store associated metadata
21: end if
22: Append per-ship result to output dictionary R
23: return R as JSON file
```

E Configuration Template

```
{
  "google-earth": {
    "service_account": {
      "type": "service_account",
      "project_id": "PUT_PROJECT_ID_HERE",
      "private_key_id": "PUT_PRIVATE_KEY_ID_HERE",
      "private_key": "PUT_PRIVATE_KEY_HERE",
      "client_email": "PUT_SERVICE_ACCOUNT_EMAIL_HERE",
      "client_id": "PUT_CLIENT_ID_HERE",
      "auth_uri": "https://accounts.google.com/o/oauth2/auth",
      "token_uri": "https://oauth2.googleapis.com/token",
      "auth_provider_x509_cert_url": "https://www.googleapis.com/oauth2/v1/certs",
      "client_x509_cert_url": "PUT_CLIENT_CERT_URL_HERE"
    }
  },
  "ais_data": {
    "base_url": "https://coast.noaa.gov/htdata/CMSP/AISDataHandler/2024/",
    "url_pattern": "AIS_2024_{date}.zip",
    "year": 2024
  }
}
```